NEXT-TOKEN PREDICTION ON WIKITEXT-103 WITH THE HIERARCHICAL SPAWN-AND-PRUNE MODEL

AN EMPIRICAL EVALUATION OF ADAPTIVE CAPACITY IN HIERARCHICAL ATTENTION

JONATHAN OTTERSPEER

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN DATA SCIENCE & SOCIETY
AT THE SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
OF TILBURG UNIVERSITY

STUDENT NUMBER

2149035

COMMITTEE

Prof. dr. Afra Alishahi

Dr. Sharon Ong

LOCATION

Tilburg University

School of Humanities and Digital Sciences

Department of Cognitive Science & Artificial Intelligence

Tilburg, The Netherlands

DATE

January 23, 2026

WORD COUNT

8767

ACKNOWLEDGMENTS

I would like to thank Professor Alishahi for supporting my ideas and helping me develop them. I would also like to thank the university for giving me the opportunity to switch academic fields and pursue a master's degree in data science. Furthermore, I would like to thank my family for instilling in me a sense of curiosity. Lastly, I would like to thank the LORD for providing me with inspiration and perseverance for this thesis.

NEXT-TOKEN PREDICTION ON WIKITEXT-103 WITH THE HIERARCHICAL SPAWN-AND-PRUNE MODEL

AN EMPIRICAL EVALUATION OF ADAPTIVE CAPACITY IN HIERARCHICAL ATTENTION

Jonathan Otterspeer

Abstract

Next-token prediction in decoder-only Transformers remains the dominant pretraining objective for large language models, but improving performance via scaling comes with diminishing returns and energy costs. This thesis investigates whether adaptive capacity control can improve accuracy under a matched compute budget by reallocating attention capacity during training. I propose a Hierarchical Spawn-and-Prune (HSAP) attention mechanism in which each attention head is equipped with a learned scalar gate that estimates utility. Low-utility heads are pruned, while high-utility heads can spawn specialized child heads. A hierarchical specialization constraint (Gaussian gating) encourages a coarse-to-fine allocation of attention. To isolate the marginal contribution of each mechanism, four compute-matched variants were trained on WikiText-103 under identical preprocessing and a shared backbone (8 heads, 12 layers, 512 model dimension): a baseline Transformer, a prune-only gated variant, a prune+spawn variant, and the full HSAP. Under equal wall-clock training, test perplexity was 32.89 for the baseline, 35.16 for the prune-only model, 34.93 for the prune+spawn model, and 33.00 for HSAP. In a context-window ablation, the Effective Context Length (95% criterion) was 256 tokens for

the baseline versus 64 for HSAP, indicating greater robustness of the HSAP under truncated context. Token-frequency analyses further showed that performance differences are regime-dependent, with HSAP's relative advantage concentrated among high-frequency tokens. Overall, HSAP does not improve full-context average next-token prediction under strict compute matching, but it changes regime behavior in ways that are relevant for long-context efficiency and deployment constraints.

Data Source/Ethics/Code/Technology (DSECT) Statement

Data Source

The author used the WikiText-103 dataset with the standard training, validation, and test splits curated by Salesforce Research from English Wikipedia "Good" and "Featured" articles (Merity et al., 2016), obtained via Hugging Face Datasets (Lhoest et al., 2021), on November 3, 2025. The corpus is released under CC BY-SA 3.0. The author cites the dataset card and original announcement. No new data were collected from human or animal participants. All figures and tables were created by the author.

Ethics

The data did not involve human participants. The WikiText-103 dataset consists of English articles and encyclopedia-style content and inherits known Wikipedia biases (for example, geography, gender, culture). The author discloses these limitations and reports results only for this domain. No additional de-identification was performed beyond Wikipedia's public release.

Code

Python version 3.11.9 (Python Software Foundation, 2024) was used to train the models. Libraries used included PyTorch version 2.10 (Paszke et al., 2019), Hugging Face Datasets version 4.4.1 (Lhoest et al., 2021), SentencePiece version 0.2.1 (Kudo & Richardson, 2018), NumPy version 2.3.5 (Harris et al., 2020), Optuna version 4.6.0 (Akiba et

al., 2019), SciPy version 1.16.3 (Virtanen et al., 2020), Matplotlib version 3.10.7 (Hunter, 2007) and tqdm version 4.67.1 (da Costa-Luis, 2019). Experiments were implemented in PyTorch with a single shared codebase covering the baseline model and all model variants. No external source code was used/copied into the project, and all implementation code was written by the author. Any AI-generated suggestions were reviewed and rewritten as needed.

Technology

The author used ChatGPT (GPT-5.2 Standard mode, GPT-5.2 Thinking mode, and GPT-5.2 Pro) for wording improvements (paraphrasing, spelling, grammar), cutting words, and organizing and summarizing the literature. It was *not* used to generate ideas for model architecture, although ideas were discussed with ChatGPT (OpenAI, 2026).

Furthermore, ChatGPT-5.2 Pro was used for drafting code snippets based on existing scripts (for example, drafting a code snippet for combining two scripts). Importantly, ChatGPT was *not* used to design the HSAP architecture, write the thesis, produce the experimental setup, conduct the error analysis, make the experimental choices or design the training procedure. All critical decisions were made by the author. In addition, the answers and code snippets from ChatGPT were not copy-pasted, and were critically reviewed and edited by the author. Typesetting was done in Microsoft Word, and no reference management software was used. Instead, the website Scribbr was used for generating drafts for in-text citations and reference list entries. The author chose to follow the APA guidelines on headings, subheadings, font size, indentation, labeling figures. However, the structure and ordering of the template are still used (i.e., placing the Acknowledgment section before the Abstract section).

All models were trained and tuned on a single RTX 5070 GPU with 12 GB VRAM using CUDA version 12.8 (Nickolls et al., 2008), combined with model offloading to an AMD Ryzen 7 processor and 32 GB of RAM for loading and processing the dataset.

Introduction

Context

Modern large language models (LLMs) are commonly trained with next-token prediction (NTP), using a decoder-only Transformer (Vaswani et al., 2017; Brown et al., 2020). Empirically, NTP loss decreases with scaling, but this process comes with diminishing returns, increasing compute and data in exchange for small performance gains (Kaplan et al., 2020; Hoffmann et al., 2022). This implies that simple scaling can become impractical. Therefore, several studies have sought to improve the efficiency of the model, rather than scaling alone.

One such direction is adaptive capacity control, which adjusts effective model capacity during training or inference. Two common approaches are pruning, removing or deactivating low-utility parameters, and conditional computation, which routes tokens through a subset of parameters (e.g., mixture-of-experts) so only part of the network is active per token (Shazeer et al., 2017; Fedus et al., 2021). Prior pruning studies suggest that many attention heads are redundant and can be removed with limited degradation (Michel et al., 2019; Voita et al., 2019). However, the benefit hinges on whether reduced computation yields faster training or higher accuracy under a fixed compute budget. Conditional computation can add routing overhead and complicate optimization without reallocating redundant capacity. Furthermore, most pruning work reduces cost by removing redundancy rather than reallocating capacity to high-utility layers or heads during training. Finally, hierarchical attention has shown strong performance (Chen et al., 2025; Liu et al., 2024; Wang et al., 2021), but has not been induced by pruning unnecessary heads and reallocating capacity to form an explicit hierarchy.

Based on this literature, this thesis contributes in three ways: (1) HSAP, an explicit spawn-and-prune capacity controller that reallocates attention-head capacity while

encouraging hierarchical specialization; (2) a controlled evaluation against a baseline decoder-only Transformer under a matched wall-clock compute budget; and (3) diagnostic analyses to characterize model behavior.

Scientific and Societal Relevance

Scientifically, this thesis advances the pruning literature by studying both capacity removal and capacity reallocation via spawning. It also identifies when hierarchical specialization emerges and how it affects next-token prediction. Compute efficiency also has societal relevance: training and serving language models consume substantial energy, and lowering compute can reduce emissions (Strubell et al., 2019; Patterson et al., 2021). Furthermore, the thesis aims to improve accuracy per unit compute under a fixed wall-clock budget by shifting redundant capacity from low-utility heads to higher-utility components. Because compute budgets limit who can train and deploy language models, better performance under fixed budgets can broaden societal access.

Research Strategy

This thesis proposes a Hierarchical Spawn-and-Prune (HSAP) Transformer, an adaptive capacity controller for attention heads. Each head receives a learnable scalar gate that estimates utility: low-utility heads are pruned, whereas high-utility heads can spawn specialist child heads that inherit and refine their parent representations. This top-down hierarchy is intended to evolve from coarse patterns to granular specialization by efficiently reallocating capacity. I evaluate on WikiText-103 (which is composed of Wikipedia articles) using the standard train/validation/test splits, comparing a baseline decoder-only Transformer with HSAP and compute-matched ablations (prune-only and prune+spawn) under a controlled, matched wall-clock budget. Both models use the same preprocessing pipeline, optimizer configuration, and logging to isolate the effect of the proposed architecture.

Although WikiText-103 is convenient for language modeling, it reflects well-known

Wikipedia biases. Internal surveys report that active editors remain predominantly male: in 2024, ~80% identified as men and 14% as women (Wikimedia Foundation, 2025). Wikipedia articles also exhibit gendered language and ingroup bias (Oeberst et al., 2020; Schmahl et al., 2020). Accordingly, results are interpreted as benchmark-specific rather than as unbiased language-modeling performance. Nevertheless, the dataset lacks reliable metadata, so these biases are discussed as context rather than measured variables. Hyperparameters are selected on the validation set, and next-token prediction quality is assessed on the held-out test set. The main research question is: Does HSAP improve next-token prediction on WikiText-103 relative to a baseline decoder-only Transformer when both are trained under the same wall-clock compute budget? This question is unpacked into sub-research questions, all evaluated under the same fixed wall-clock budget and identical preprocessing, training, and testing pipeline.

1. Does pruning via a learned gate improve next-token prediction performance significantly relative to a baseline Transformer?
2. What is the marginal effect of adding spawning beyond pruning on next-token prediction (versus pruning-only and baseline Transformer)?
3. What is the marginal effect of enforcing hierarchical specialization beyond pruning+spawning on next-token prediction (versus pruning+spawning, pruning-only, and baseline Transformer)?
4. How does training efficiency differ across the four variants (baseline Transformer, pruning-only, pruning+spawning, full HSAP), measured as steps and wall-clock time to reach a target validation loss?
5. Across variants, how do prediction errors vary by token frequency and context length, how does probabilistic calibration differ, and are performance differences concentrated in particular frequency bins or context lengths?

6. How sensitive is each variant to attention-head and layer ablations, measured by the increase in test cross-entropy and test perplexity when removing heads or individual layers?

Findings

In compute-matched WikiText-103 experiments, the Hierarchical Spawn-and-Prune (HSAP) architecture did not improve average full-context next-token prediction over a strong baseline Transformer under the same wall-clock budget. Learned-gate pruning substantially degraded performance, with spawning without hierarchical specialization providing partial recovery. Enforcing hierarchical specialization recovered performance, returning HSAP to baseline-level performance while outperforming the other gated variants. Beyond aggregate loss, the variants differed in robustness and probabilistic behavior: HSAP degraded less under reduced effective context, was better at modelling more frequent tokens and showed calibration differences from the baseline. Overall, under strict compute matching, adaptive capacity control appears more promising for robustness and context efficiency than for lowering full-context perplexity.

Literature Review

Next-Token Prediction and the Decoder-Only Transformer Baseline

Next-token prediction (NTP) is the dominant LLM pretraining objective (He & Su, 2024). Because NTP gains correlate with general model capability, it is a useful testbed for studying accuracy–efficiency trade-offs (Lin et al., 2025; Tack et al., 2025; Wang et al., 2024). In NTP, models predict the next token from prior tokens and are optimized with cross-entropy. Perplexity is the standard evaluation metric (Braverman et al., 2019; Jurafsky & Martin, 2025, pp. 45–47). The common baseline is the decoder-only Transformer with causal self-attention (Vaswani et al., 2017; Radford et al., 2019): stacked decoder blocks with multi-head self-attention, position-wise feed-forward layers, residual connections, and layer

normalization. Evaluation typically uses benchmarks with standard splits, including WikiText-103, a 103M-token dataset of English Wikipedia Good and Featured articles (Merity et al., 2016) that is feasible under limited budgets and supports long-range dependency modeling (Dai et al., 2019). Accordingly, results are often reported against the decoder-only Transformer baseline (Beltagy et al., 2020; Dai et al., 2019; Vaswani et al., 2017; Zhu & Soricut, 2021), and the next section summarizes representative WikiText-103 perplexities as reference points.

The State-of-the-Art on WikiText-103

Transformer-XL introduced segment-level recurrence to extend context length and reported a test perplexity of 18.3 on WikiText-103 (Dai et al., 2019). Baevski and Auli (2019) used adaptive input representations and reported 18.7. Retrieval augmentation can reduce perplexity further. Yogatama et al. (2021) combined long-term memory during training with kNN interpolation at inference and reported 17.6. Alon et al. (2022) also used retrieval augmentation and reported 16.08.

Pretraining on other corpora and then fine-tuning or zero-shot testing on WikiText-103 can improve performance. For example, Sathe and Sarawagi (2024) fine-tuned GPT-2, GPT-Neo, and OPT models on WikiText-103 and reported 15.679. Building on this, Chen et al. (2023) pretrained GPT-2 on the WebText dataset and evaluated it zero-shot on WikiText-103, reporting a perplexity of 33.0. Nevertheless, reported perplexities are sensitive to modeling choices such as tokenization scheme and evaluation protocol. These values are therefore difficult to compare directly across papers. They are best treated as contextual reference points, not as a single leaderboard. This sensitivity motivates compute-matched, controlled comparisons against a fixed decoder-only baseline, using ablations to isolate architectural effects. With this context in mind, the next section reviews scaling and the motivation for compute-efficient architectures.

Model Size, Compute and Scaling Laws

Within decoder-only Transformers, empirical scaling laws characterize how cross-entropy changes as model size, dataset size, and/or training compute increase while the architecture family is held fixed. Scaling generally improves performance predictably, but with diminishing returns. Hestness et al. (2017) find power-law decreases in test and generalization error as model and dataset size grow. Kaplan et al. (2020) likewise show that cross-entropy falls when scaling parameters, data, or compute, and that the most effective lever depends on the compute–data regime. However, these power-law relationships can break down in some regimes (Caballero et al., 2022; Sorscher et al., 2022).

Against this backdrop, Hoffmann et al. (2022) argue that many models are undertrained for their parameter count and, under a fixed compute budget, derive an allocation rule in which model and data should scale roughly equally. Using this rule, they trained Chinchilla, which outperformed much larger models such as GPT-3 175B. More recent work suggests that scaling is not the only driver and that architecture matters. Bian et al. (2025) propose a conditional scaling law that treats architecture as an explicit factor alongside model and dataset size. With the architecture predicted by their framework, they report 2.1% higher accuracy and 42% greater inference throughput than LLaMA 3.2. Taken together, these findings motivate targeted, compute-efficient architectural mechanisms that complement scaling**.**

Adaptive Capacity Control

The decoder-only Transformer is the dominant NTP baseline (Radford et al., 2019; Vaswani et al., 2017). Scaling can improve performance but increases cost with diminishing returns. Since architectural choices can also yield gains (Bian et al., 2025), this thesis studies adaptive capacity control under a fixed compute budget: selectively allocating capacity to improve accuracy per unit compute. Most LLMs are dense: every attention head and neuron

is evaluated for every token, regardless of difficulty or informativeness, making the design simple and reliable but potentially wasteful on easy tokens. Sparsity addresses this by activating only a subset of parameters per input, either statically or dynamically. Static sparsity (pruning) permanently removes underutilized parameters or structures via an importance criterion (Behnke & Heafield, 2020; Li et al., 2023; Michel et al., 2019; Voita et al., 2019). Dynamic sparsity uses conditional computation with a learned gate or router to activate only a subset of experts, feed-forward blocks, or heads per token, allocating more capacity to hard tokens and less to easy ones.

Routing and Mixture-of-Experts

Mixture-of-experts (MoE) architectures are a form of dynamic sparsity in which a gating network routes inputs to a subset of expert subnetworks. Shazeer et al. (2017) showed that sparsely gated MoE layers can substantially increase parameter count while keeping compute approximately constant. Switch Transformer simplified routing with top-1 expert selection and achieved faster pretraining than dense T5 models under comparable compute budgets (Fedus et al., 2021). For decoder-only language modeling, GLaM likewise showed that conditional computation can deliver strong performance with lower energy use than dense models at similar capability levels (Du et al., 2021). Nevertheless, routing introduces additional architectural choices and systems complexity. Moreover, MoE reduces computation through selective activation rather than parameter removal, so inactive experts remain stored and can impose memory overhead, which can limit practical efficiency gains under fixed budgets. These trade-offs make structured pruning an attractive alternative when the goal is to remove capacity explicitly under fixed-budget constraints.

Pruning

Pruning reduces model capacity by removing structured components that contribute little to predictive performance. In Transformers, pruning can target attention heads, feed-

forward blocks, or individual weights, and a consistent finding is that substantial portions of models can be removed with little or no performance loss (Behnke & Heafield, 2020; Li et al., 2023; Michel et al., 2019). Focusing on attention heads, multiple approaches support high pruning rates with limited degradation. Voita et al. (2019) showed—using attention-statistics-based pruning—that most heads could be removed with minimal impact on machine translation quality. Similarly, Michel et al. (2019) reported that many heads can be pruned on WMT14 En–De with negligible impact on BLEU. Together, these results suggest that head pruning can preserve competitive performance while reducing active attention computation.

Head-level pruning is a natural structured unit for studying redundancy because it removes attention capacity without deleting entire layers. Relative to unstructured weight pruning, it yields structured sparsity that more directly reduces attention computation and is easier to evaluate in controlled comparisons; relative to pruning whole layers or feed-forward blocks, it offers finer-grained control by reducing attention capacity without removing entire components. This makes head pruning well suited for studying adaptive capacity control and its interaction with specialization and long-range dependencies.

Methodologically, prior head-pruning work typically falls into three groups: (1) distillation-based approaches that train a smaller student to match a larger teacher while pruning heads (Li et al., 2023); (2) attention-statistics heuristics that use proxies derived from attention patterns (Behnke & Heafield, 2020; Voita et al., 2019); and (3) gradient-based or learned selection approaches that estimate importance more directly, either via loss sensitivity (Michel et al., 2019) or via learnable gates that induce structured sparsity (Li et al., 2021). However, these methods can introduce compute overhead through gating or importance-ranking procedures.

In summary, the literature establishes attention-head pruning as a well-motivated mechanism for removing underutilized components while maintaining competitive

performance. At the same time, pruning addresses removal rather than reallocation: it does not specify how freed attention capacity should be redeployed. Hierarchical attention therefore provides a complementary perspective on where to allocate capacity efficiently, and is reviewed next as a candidate principle for redeploying attention capacity.

Hierarchical Attention

Hierarchical attention addresses the quadratic cost of flat self-attention by partitioning attention into local patterns and a coarser global component. Longformer combines sliding-window attention with global tokens and achieved strong performance on long-document benchmarks (Beltagy et al., 2020), improving leaderboard F1 from 78.3 to 81.9 on WikiHop and from 73.3 to 77.3 on TriviaQA. In a more explicitly hierarchical design, H-Transformer-1D decomposes attention into a hierarchical tree, reducing runtime and memory while maintaining competitive performance; on the One Billion Word dataset, it achieved perplexity 20.25 versus 24.8 for the baseline Transformer.

More recently, hierarchical attention has been extended to multiscale autoregressive decoders. MEGABYTE uses global attention across patches with local modeling within patches for long byte sequences (Yu et al., 2023), reporting perplexity 36.4 versus a baseline byte-level Transformer. Autoregressive U-Net pools raw bytes into progressively coarser representations so higher levels capture broader dependencies (Videau et al., 2025); AU-Net achieved MMLU accuracy 31.7 versus 27.0 for the baseline Transformer. Overall, these results support hierarchical attention as a compute-efficient mechanism for long-range language modeling under constrained budgets.

Summary of the Literature

This chapter positioned the decoder-only Transformer as the baseline architecture for NTP, where models minimize cross-entropy and are commonly evaluated via perplexity on benchmark datasets such as WikiText-103. Scaling-law research further shows that increasing parameters, data, and compute improves performance but with diminishing returns; accordingly, this thesis targets accuracy under a fixed compute budget through architectural choices.

From the literature, two primary efficiency levers emerge. First, compute can be reallocated via routing-based dynamic sparsity (e.g., mixture-of-experts). Second, redundant components can be removed via structured pruning. Although routing can improve efficiency, it retains inactive parameters and introduces routing and systems complexity, which can limit practical compute reductions under fixed budgets and interpretability. By contrast, structured pruning removes capacity explicitly, and multiple studies report that substantial fractions of attention heads can be removed with little effect on performance.

At the same time, pruning addresses removal rather than redeployment. Empirical evidence remains limited on training-time approaches that combine hierarchical structure with systematic pruning and explicit regrowth under a fixed compute budget, leaving it unclear whether capacity can be removed and effectively redeployed without degrading predictive performance. To my knowledge, this combination has not been studied as a unified adaptive controller that jointly removes and reallocates attention capacity during training. This gap motivates the present work.

Methodology and Experimental Setup

Methodology

Based on the literature reviewed, I introduce the core components and training procedure of the HSAP model. Standard multi-head self-attention (Vaswani et al., 2017) is

assumed to be well-known. A complete list of symbols is provided in Appendix A.

The Gated Attention Mechanism

Let $x_l \in \mathbb{R}^{T_{\text{seq}} \times d_{\text{model}}}$ denote the input sequence to Transformer layer l, and let layer l contain H_l attention heads indexed by $i \in \{1, \dots, H_l\}$. I identify a head by $h = (l, i)$. Each attention head is represented as the function:

$$f_{l,i}(x_l; \theta_{l,i}),$$

where $\theta_{l,i}$ denotes the parameters of head i in layer l. Instead of concatenating head outputs, HSAP forms the attention sublayer output by a gated sum:

$$y_l(x_l) = \sum_{i=1}^{H_l} g_{l,i} f_{l,i}(x_l; \theta_{l,i})$$

where $g_{l,i} \in (0,1)$ is a learned scalar gate for head (l, i).

Each gate is parameterized by an unconstrained scalar $a_h \in \mathbb{R}$ with a logistic sigmoid function:

$$g_h = \sigma(a_h).$$

Specialization of Attention Heads

To promote hierarchical specialization, the HSAP model further introduces a Gaussian gate. For each head h, we learn a center (prototype) $c_h \in \mathbb{R}^{d_{\text{head}}}$ and a width $\sigma_h > 0$, where d_{head} is the dimensionality of the per-head query vectors. Let $q_{l,h,t} \in \mathbb{R}^{d_{\text{head}}}$ be the query vector produced by head h in layer l at token position t. We compute Gaussian activation:

$$w_{l,h,t} = \exp\left(-\frac{|q_{l,h,t} - c_h|^2}{2\sigma_h^2}\right)$$

The width σ_h controls how narrowly the head specializes. Let $y_{l,h,t}$ denote the usual attention output produced by head h at token t. HSAP head output becomes:

$$\tilde{y}_{l,h,t} = g_h \, w_{l,h,t} \, y_{l,h,t}.$$

To prevent unfavorable widths, σ_h is constrained to remain positive with a lower bound $\sigma_{\min} > 0$. Overall, the scalar gate g_h controls global head utility, while the Gaussian gate $w_{l,h,t}$ encourages heads to increasingly specialize in representation space, yielding a coarse-to-granular, top-down allocation of attention.

Sparsity Regularization on Gates

To encourage sparsity, the HSAP model adds an L1 penalty on the scalar gates to the task loss. Given a training dataset $D = \{(x^{(j)}, y^{(j)})\}_{j=1}^{N}$, the total objective is:

$$L_{total} = \frac{1}{N} \sum_{j=1}^{N} l_{task}\left(f_{model}\left(x^{(j)}\right), y^{(j)}\right) + \lambda_{sparse} \sum_{h \in T} |g_h|$$

Here, $f_{\text{model}}(\cdot)$ denotes the model forward pass, $l_{\text{task}}(\cdot)$ the task loss, $\lambda_{\text{sparse}} \geq 0$ the sparsity coefficient, and T the set of all gated attention heads included in the penalty (i.e., across all layers).

Pruning of Low-Utility Heads

Structural changes are applied at a predefined set of global training steps $S = \{s_1, \ldots, s_K\}$. At each structural step s_k, we compute a within-layer normalized gate score to make thresholds comparable across layers:

$$\nu_h = \frac{g_h}{\max\limits_{i=1,\ldots,H_l} g_{l,i}}$$

for head $h = (l, i)$ in layer l.

Given a pruning threshold $\tau_{\text{prune}} \in (0,1)$, the set of heads pruned from layer l at step s_k is:

$$P_l^{(s_k)} = \{\, h = (l,i) \in A_l^{(s_k)} \mid \nu_h < \tau_{\text{prune}} \,\}.$$

All heads in $P_l^{(s_k)}$ are permanently removed, and training continues with the remaining heads until the next structural step.

Spawning Child Heads from Parent Heads

In addition to pruning, HSAP can allocate capacity by splitting high-utility heads into specialized child heads. Let $A_l^{(s_k)}$ denote the set of active (non-pruned) heads in layer l at structural step s_k. Given a spawning threshold $\tau_{\text{spawn}} \in (0,1)$, we define a set of eligible parents:

$$Spawn_l^{(s_k)} = \{\, h \in A_l^{(s_k)} \mid \nu_h \geq \tau_{\text{spawn}} \,\}.$$

To enforce a clean comparison under a fixed global head budget, spawning is constrained by the number of heads pruned at step s_k. Let:

$$R^{(s_k)} := \sum_{l=1}^{L} \left| P_l^{(s_k)} \right|$$

denote the number of head slots freed by pruning at step s_k. We then select

$$Spawn^{(s_k)} = \cup_{l=1}^{L} Spawn_l^{(s_k)}$$

as the top-$R^{(s_k)}$ eligible parents ranked by ν_h. For each selected parent head $h \in \widetilde{Spawn}^{(s_k)}$, we create $K_{child} = 1$child head $h_{child}(h)$. Each child is initialized by the parent with a small perturbation:

$$\theta_{h_{\text{child}}(h)} = \theta_h + \epsilon.$$

where ϵ is a small random perturbation (e.g., Gaussian noise with small variance). Each child is assigned a fixed small initial gate value g_{init}, while the parent retains its original gate:

$$g_{h_{\text{child}}(h)} = g_{\text{init}}.$$

Overall Procedure

Training the full HSAP model alternates between standard gradient-based optimization of all parameters (including θ_h and a_h) under L_{total}, and structural updates at steps $S = \{s_1, \ldots, s_K\}$. At each structural step s_k, the procedure is:

1. Continue training up to step s_k under L_{total}.

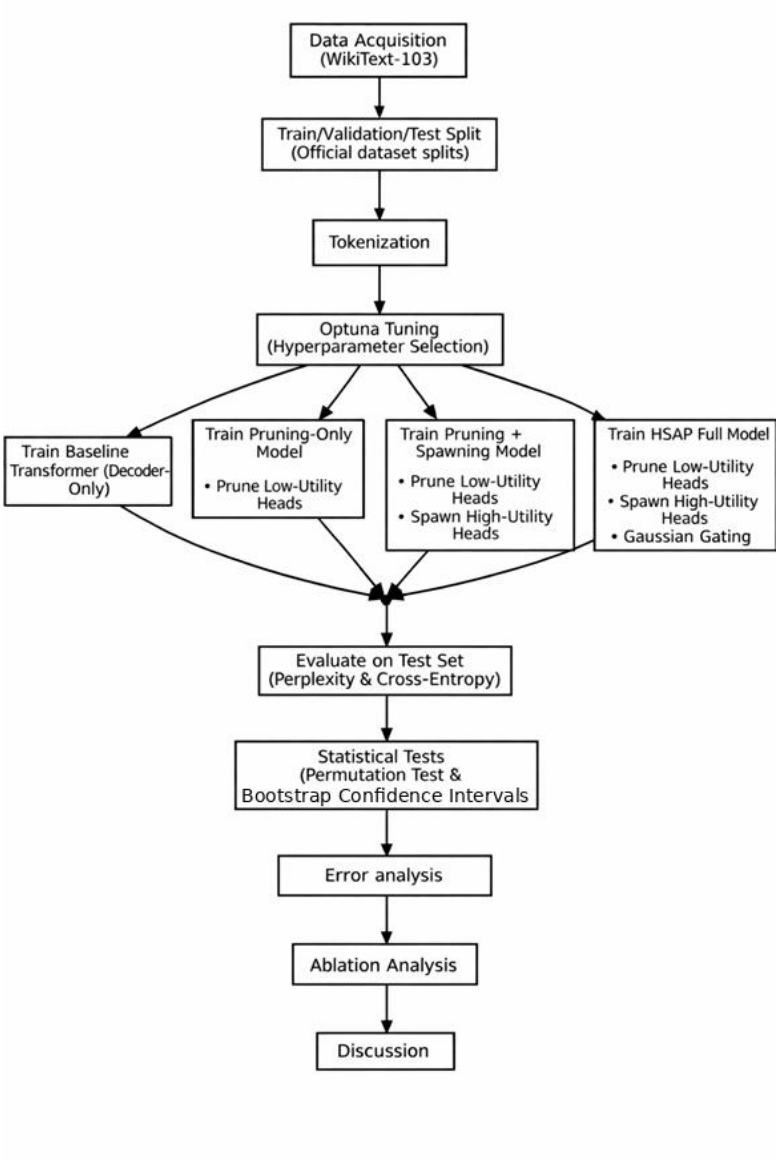
2. Compute gate values g_h and normalized scores ν_h.
3. Prune heads with $\nu_h < \tau_{\text{prune}}$.
4. Spawn child heads for parents with $\nu_h \geq \tau_{\text{spawn}}$.
5. Newly spawned heads specialize (via Gaussian gating).

Pipeline

The HSAP was evaluated in a controlled, compute-matched comparison of four variants trained with an identical protocol. Starting with a decoder-only Transformer baseline (Vaswani et al., 2017), I then added a pruning-only model that used scalar head gates with sparsity regularization to prune low-utility heads. A prune+spawn variant further allowed high-utility heads to spawn child heads to reallocate capacity. The full HSAP added a Gaussian gate that encouraged spawned heads to specialize in subregions of representation space. Hyperparameters were tuned with Optuna, and models were trained on WikiText-103 for next-token prediction and evaluated primarily on the held-out test set. Performance differences were analyzed with a permutation test and bootstrap confidence intervals. I also analyzed the effect of token frequency, the effect of context ablation, the probabilistic calibration of each model, and head/layer contributions via ablation. Figure 1 depicts the data science pipeline.

Figure 1

A Flowchart of the Data Science Pipeline for Evaluating the Four Model Variants



Experimental Setup

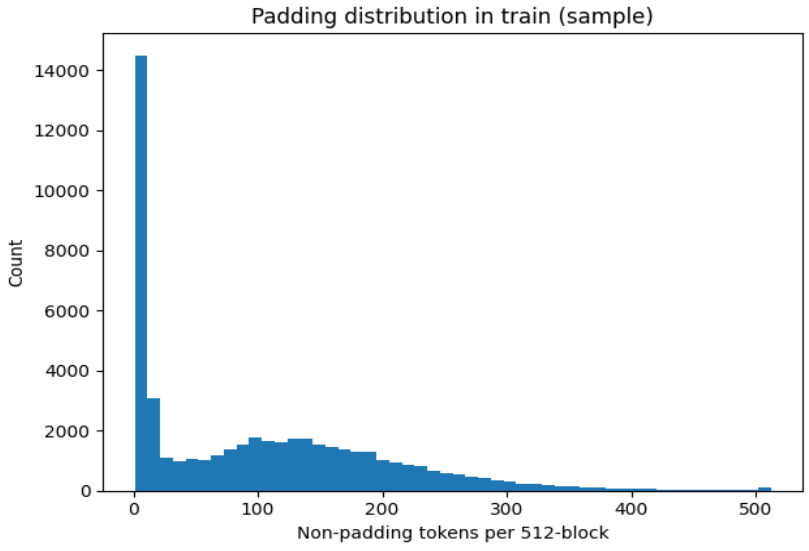
WikiText-103 (Merity et al., 2016) was used for next-token prediction (NTP). Released by Salesforce Research, it comprises English Wikipedia "Good" and "Featured" articles and contains 103.7 million tokens, making it suitable for evaluating long-range dependencies and because it is large enough to detect meaningful differences while remaining feasible for tuning and training multiple variants (Baevski & Auli, 2019). I used its fixed

train/validation/test splits for comparability with prior NTP work (Dai et al., 2019; Merity et al., 2016). The training set contains 1,801,350 segments (103,227,021 tokens), the validation set 3,760 segments (217,646 tokens), and the test set 4,358 segments (245,569 tokens). Data were accessed via Hugging Face Datasets (Lhoest et al., 2021) from Salesforce/wikitext (wikitext-103-raw-v1) under the Creative Commons Attribution–Share Alike 3.0 license.

Text was tokenized with SentencePiece subwords (Kudo & Richardson, 2018) using a 32,000-token vocabulary, which limits the vocabulary size, reduces the memory footprint of the embedding and output projection layers, and mitigates open-vocabulary issues (Sennrich et al., 2016). Initially, I formed fixed-length 512-token sequences by splitting longer segments into non-overlapping blocks and padding/masking shorter segments. However, Figure 2 shows this yielded many sequences with only 10–50 non-padding tokens. I therefore used continuous-stream tokenization by appending an EOS token after each segment and splitting the stream into non-overlapping 512-token sequences. Aside from tokenization, no additional preprocessing was applied.

Figure 2

Distribution of Non-Padding Tokens per 512 Token Block in the Training Data



Model Ablations

To estimate the marginal effect of each HSAP component, I evaluated four nested decoder-only Transformer variants under an identical backbone and training protocol (Table 1). The baseline is a Transformer with standard multi-head self-attention (Vaswani et al., 2017). The pruning-only variant added a scalar gate per attention head with an $L1$ sparsity penalty and removed low-utility heads. The pruning-and-spawning variant further allowed high-utility parent heads to spawn child heads to reallocate freed capacity during training. The full HSAP variant added a token-dependent Gaussian gate that encouraged spawned heads to specialize.

All four model variants shared the same backbone (Table 1) so differences can be attributed to HSAP mechanisms rather than overall capacity. A 512-token context was chosen to balance usable context with compute cost. All models minimized cross-entropy, and perplexity was reported for model comparison; cross-entropy is a proper scoring rule that penalizes high-confidence errors and is the standard objective in modern NTP work (Jurafsky & Martin, 2025, pp. 45–47). Batch size was fixed at 8 due to computational constraints. The feed-forward sublayer used GELU, pre-LayerNorm was applied for optimization stability, and dropout mitigated overfitting (Hendrycks & Gimpel, 2016; Srivastava et al., 2014; Xiong et al., 2020). Optimization used AdamW with warm-up and cosine learning-rate decay, consistent with findings on warm-up sensitivity and cosine schedules (Loshchilov & Hutter, 2016; Loshchilov & Hutter, 2019; Xiong et al., 2020).

A key design choice was to implement pruning and spawning by shrinking and rebuilding projections rather than masking heads. When heads were pruned or spawned, the attention module rebuilt the packed QKV and output projections, so pruning yielded real compute savings under a fixed wall-clock budget instead of masking head outputs, which does not reduce matrix-multiplication cost.

Table 1

The Training Protocol and Backbone of All Four Model Variants

Setting	Value	Notes
Model family	Decoder-only Transformer LM	Shared across all four variants
Decoder layers (L)	12	Fixed for comparability
Model dimension (d_model)	512	Fixed for comparability
FFN dimension (d_ff)	2,048	Fixed for comparability
Initial heads per layer	8	All variants start with the same head count
Maximum sequence length	512	Matches positional embedding length
Optimizer	AdamW	Used consistently across trials
LR schedule family	Warm-up + cosine decay	Same schedule family across variants
Batch size	8	Fixed because of compute
Optuna objective	Validation cross-entropy	—
Loss function	Cross-entropy loss	Standard loss function
Evaluation metric	Perplexity	The exponentiation of cross-entropy

Note. Em dash (—) indicates not applicable.

Hyperparameter Tuning

Hyperparameters were divided into two categories: a shared backbone (Table 1), whose parameters were fixed, and tunable hyperparameters (Tables 2 and 3). The tunable hyperparameters of the four model variants and the search space per hyperparameter (shown in Table 2) were tuned with Optuna (Akiba et al., 2019).

Table 2

Tunable Hyperparameters and the Search Space for the Four Models

Hyperparameter	Search space (Optuna)	Baseline	Prune-only	Prune+spawn	Full HSAP
Learning rate	log-uniform [1×10−5, 1×10−3]	✓	✓	✓	✓
Weight decay	log-uniform [1×10−6, 1×10−2]	✓	✓	✓	✓
Gradient clipping	uniform [0.5, 2.0]	✓	✓	✓	✓
Dropout	uniform [0.0, 0.3]	✓	✓	✓	✓
Label smoothing	uniform [0.0, 0.2]	✓	✓	✓	✓
Warm-up steps	categorical {2,000, 4,000, 6,000, 8,000, 10,000}	✓	✓	✓	✓
Sparsity strength	log-uniform [1×10−6, 1×10−2]	—	✓	✓	✓
Pruning threshold	uniform [0.05, 0.6]	—	✓	✓	✓
Structural step interval	categorical {2,000, 4,000, 6,000, 8,000, 10,000}	—	✓	✓	✓
Spawning threshold	uniform [max(tau_prune + 0.05, 0.6), 0.99]	—	—	✓	✓
Layer head cap	categorical {8, 16, 24, 32, 48, 64, 96, 128}	—	—	✓	✓
Child perturbation	uniform [0.0, 0.2]	—	—	✓	✓
Child initial gate	uniform [0.01, 0.3]	—	—	✓	✓
Gaussian shrink factor	uniform [0.3, 0.9]	—	—	—	✓

Note. A check mark ✓ indicates applicable to that model variant. An em dash (—) indicates not applicable.

The tuning space mirrored the nested model design, so each variant introduced a small set of additional hyperparameters. For the baseline Transformer, I tuned learning rate, weight decay, gradient clipping, dropout, and label smoothing. The pruning-only variant added sparsity controls (sparsity coefficient, pruning threshold, and structural-step interval). The pruning-and-spawning variant added spawning hyperparameters (spawning threshold, maximum heads-per-layer cap, and child-initialization controls). The full HSAP variant added Gaussian specialization controls, most notably the child shrink factor, which determines how much narrower a child head is relative to its parent.

Each Optuna trial ran for 1.5 hours on a single RTX 5070, with 20 trials per variant, evaluated on the validation set. The search space and Optuna-selected values are reported in Table 3. Using the selected hyperparameters, each model was then trained for 6 hours on the same GPU to learn nontrivial patterns without substantially increasing compute or undermining comparability with the tuning runs. Because final training was four times longer than tuning, I scaled the structural interval by four to keep the mechanisms aligned with training progress, while holding all other hyperparameters constant. Furthermore, a global minimum of 32 heads and a minimum of 1 head per layer were enforced to prevent catastrophic degradation.

During tuning, the Optuna-selected configuration for the prune+spawn variant produced no structural updates in the final runs: no heads were pruned, which prevented spawning because spawning requires freed head slots. For the final prune+spawn run, I therefore reused the HSAP controller hyperparameters $\mathrm{tau}_{\mathrm{prune}}$, $\mathrm{tau}_{\mathrm{spawn}}$, and $\mathrm{lambda}_{\mathrm{sparse}}$, while keeping the remaining prune-and-spawn hyperparameters at their Optuna-selected values. This ensures that the prune+spawn remained a meaningful ablation of capacity reallocation rather than a structurally static gated model, and it supports a

controlled comparison with and without hierarchical specialization. Prune+spawn and HSAP shared the same controller thresholds and sparsity strength. Consequently, any difference in the number of spawned heads that remain active was interpreted as an effect of specialization rather than controller calibration.

Table 3

Tunable Hyperparameters and the Optimal Values across Model Variants

Hyperparameter	Optimal: Baseline	Optimal: Prune-only	Optimal: Prune+spawn	Optimal: Full HSAP
Learning rate	9.848×10^{-4}	9.671×10^{-4}	5.752×10^{-4}	9.671×10^{-4}
Weight decay	0.008	1.516×10^{-5}	8.355×10^{-4}	2.679×10^{-5}
Gradient clipping	1.434	0.800	0.880	1.898
Dropout	0.005	0.011	0.095	0.034
Label smoothing	0.144	0.164	0.081	0.011
Warm-up LR (and prune)	6,000	4,000	6,000	8,000
Sparsity strength	—	2.554×10^{-4}	1.163×10^{-6}	3.663×10^{-4}
Pruning threshold	—	0.598	0.245	0.598
Structural step interval	—	6,000 (24,000)	6,000 (24,000)	4,000 (16,000)
Spawning threshold	—	—	0.704	0.982
Layer head cap	—	—	16	24
Child perturbation	—	—	0.050	0.109
Child initial gate	—	—	0.060	0.079
Gaussian shrink factor	—	—	—	0.699

Note. An em dash (—) indicates not applicable. Values in parentheses indicate the settings used for the 6-hour final runs (scaled 4× from the 1.5-hour tuning trials). Warm-up LR denotes the learning-rate warm-up for all four models and the warm-up preceding pruning for the three non-baseline models.

Finally, Python version 3.11.9 (Python Software Foundation, 2024) was used for coding. Hugging Face Datasets version 4.4.1 (Lhoest et al., 2021) was used for loading the dataset, NumPy version 2.3.5 (Harris et al., 2020) was used for matrix multiplication, SentencePiece version 0.2.1 (Kudo & Richardson, 2018) was used for tokenization, Optuna version 4.6.0 (Akiba et al., 2019) was used for hyperparameter tuning, SciPy version 1.16.3 (Virtanen et al., 2020) and Matplotlib version 3.10.7 (Hunter, 2007) were used for post-hoc analysis, tqdm version 4.67.1 (da Costa-Luis, 2019) was used for progress monitoring, and PyTorch version 2.10 (Paszke et al., 2019) with CUDA version 12.8 (Nickolls et al., 2008) was used for training. Model sheets can be found in Appendix B.

Evaluation Measures

To contextualize the results, I plotted test-set token frequencies using a Zipf plot (Zipf, 1949). Out-of-sample next-token prediction performance was evaluated on the WikiText-103 test set using cross-entropy and perplexity. Training progression was monitored by validating every 10,000 optimization steps: validation cross-entropy was compared to training cross-entropy, and final training cross-entropy was compared to validation and test cross-entropy.

Each variant was trained once, so test-set differences were assessed with paired, block-level permutation tests. Cross-entropy was computed per 512-token test block and paired differences were formed. The null distribution was generated by randomly swapping model assignments within blocks and 10,000 permutations were run per comparison with two-sided p-values reported (Good, 2005). All pairwise comparisons were tested and p-values were corrected using Holm–Bonferroni at $\alpha = .05$ (Holm, 1979). Uncertainty was quantified with a circular moving-block bootstrap (10,000 resamples), reporting 95% confidence intervals for the mean paired cross-entropy difference (Politis & Romano, 1992). Effect sizes were reported as relative perplexity improvement and Cohen's $d_{(z)}$ (Cohen, 1988;

Lakens, 2013).

As a time-to-quality metric, I reported the first validation step and wall-clock time at which validation cross-entropy fell below 3.5, enabling efficiency comparisons under a shared schedule.

For error analysis, I related token frequency to test loss by plotting token frequency against mean test cross-entropy, computing Spearman's rank correlation (Spearman, 1904), and testing pairwise differences via permutation tests. Token frequencies were also grouped into bins to analyze how cross-entropy varies across frequency ranges and model variants. In addition, I assessed token-level calibration using reliability diagrams and expected calibration error (ECE; Guo et al., 2017), and measured context sensitivity by ablating context length and evaluating the impact on performance.

Finally, a post hoc ablation study removed individual attention heads at inference time by zeroing their outputs, and approximated layer ablations by zeroing the outputs of all heads in a layer; the resulting increases in cross-entropy and perplexity were measured (Michel et al., 2019).

Results

Model Performance Under Equal Compute Budget

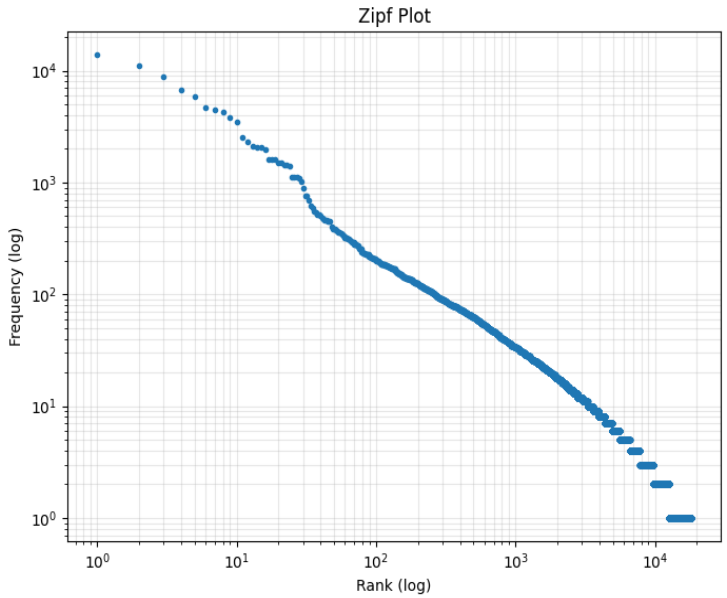
Below, I report the held-out test performance of all models trained under an equal training compute budget. Note that in the figures, *vanilla* means the baseline Transformer and in text and tables, *baseline* denotes the baseline Transformer. Also, in figures and tables, *baseline_prune* denotes the prune-only variant, *scalar_prune_spawn* denotes the prune+spawn variant, and *hsap_full* denotes the full HSAP model. Also, pruning-based variants denote *scalar_prune_spawn* and *baseline_prune*. Furthermore, CE denotes cross-entropy loss and PPL denotes perplexity. Because controller hyperparameters were standardized to ensure structural updates occurred, prune+spawn should be interpreted as a

mechanistic ablation rather than as a best-case model. Although prune+spawn and HSAP triggered pruning at comparable rates under the shared controller thresholds, fewer spawned heads remained active by the end of training in the prune+spawn model than in the HSAP, suggesting that in the absence of a specialization mechanism, newly introduced heads were less likely to become high-utility.

Before analyzing the test results, I explored the test set token frequency distribution. Figure 3 reports the Zipf distribution of token counts on log–log axes. The distribution is strongly heavy-tailed, indicating that most token types occur infrequently while a small subset accounts for a large share of instances. This motivates the frequency-binned error analysis to assess how model performance varies across token frequencies.

Figure 3

Zipf Plot of Test-Set Token Counts



To address sub-RQs 1–3 on compute-matched performance, I report held-out test performance alongside the corresponding training and validation cross-entropy (Table 4). All variants learned substantial next-token structure relative to a uniform random-guess baseline,

which would yield a cross-entropy of 10.37. For *baseline*, *baseline_prune*, and *scalar_prune_spawn*, best training cross-entropy was higher than validation and test cross-entropy, which was expected because the training objective included regularization such as label smoothing and dropout. By contrast, *hsap_full* reached a much lower best training cross-entropy (2.91) while its best validation cross-entropy was 3.455 and its test cross-entropy was 3.497, giving the largest train–evaluation gap among the four models. All models achieved nontrivial held-out performance, with test perplexities ranging from 32.89 to 35.16. The context-length analysis below further showed cross-entropy decreased as more context was available, supporting the conclusion that the models learned meaningful next-token dependencies.

Table 4

Best Training Cross-Entropy, Best Validation Cross-Entropy, Test Cross-Entropy, and Test Perplexity

Model	Best training CE	Best validation CE	Test CE	Test PPL
baseline	3.916	3.450	3.493	32.89
baseline_prune	4.196	3.511	3.560	35.16
scalar_prune_spawn	3.724	3.519	3.554	34.93
hsap_full	2.910	3.455	3.497	33.00

Table 5 reports held-out test performance with 95% confidence intervals to quantify uncertainty. On the test set, *baseline* achieved the lowest cross-entropy (3.493) and perplexity (32.89). The *hsap_full* result was very close (cross-entropy 3.497; perplexity 33.00). The other two models performed worse, with *scalar_prune_spawn* reporting cross-entropy 3.554 (perplexity 34.93) and *baseline_prune* reporting cross-entropy 3.560 (perplexity 35.16). The 95% confidence intervals were of similar width across variants, indicating comparable uncertainty, and they overlapped across the top models. Therefore, Table 6 reports pairwise permutation tests to assess differences more rigorously.

Table 5

Test Performance With Bootstrap Confidence Intervals

Model	Test CE	CE 95% *CI* (lower)	CE 95% *CI* (upper)	Test *PPL*
baseline	3.493	3.450	3.536	32.89
baseline_prune	3.560	3.516	3.603	35.16
scalar_prune_spawn	3.554	3.511	3.596	34.93
hsap_full	3.497	3.452	3.539	33.00

Note. CI denotes confidence interval.

Table 6 reports all pairwise permutation tests with Holm–Bonferroni correction and Cohen's $d_{(z)}$ as effect size. ΔPPL% normalizes ΔPPL by the reference model's perplexity for readability. The *baseline* significantly outperformed both pruning-based variants, with large effects for *baseline* vs *baseline_prune* ($d_{(z)} = -1.42$, $p < .001$) and *baseline* vs *scalar_prune_spawn* ($d_{(z)} = -1.06$, $p < .001$). In contrast, *baseline* and *hsap_full* did not differ significantly ($d_{(z)} = -0.06$, $p = .164$), indicating that their test cross-entropy was statistically indistinguishable under this test. Both pruning-based variants performed significantly worse than *hsap_full*, with large effects for *baseline_prune* vs *hsap_full* ($d_{(z)} = 1.13$, $p < .001$) and *scalar_prune_spawn* vs *hsap_full* ($d_{(z)} = 1.14$, $p < .001$). Between the pruning variants, *scalar_prune_spawn* slightly outperformed *baseline_prune* ($d_{(z)} = 0.10$, $p = .036$), although the effect size was small. Consistent with these results, *baseline_prune* also had the highest test perplexity, which may reflect an aggressive pruning schedule that reduced the number of active attention heads.

Table 6

Pairwise Permutation-Test Comparisons With Holm–Bonferroni Correction and Effect Sizes

Comparison (A vs B)	ΔPPL	Δ PPL (%)	Cohen's $d_{(z)}$	p (unadjusted values)	p (Holm–Bonferroni)
baseline vs baseline_prune	-2.27	6.46%	-1.42	$p < .001$	$p < .001$
baseline vs scalar_prune_spawn	-2.04	5.84%	-1.06	$p < .001$	$p < .001$
baseline vs hsap_full	-0.11	0.33%	-0.06	$p = .164$	$p = .164$
baseline_prune vs scalar_prune_spawn	0.23	-0.66%	0.10	$p = .018$	$p = .036$
baseline_prune vs hsap_full	2.16	-6.55%	1.13	$p < .001$	$p < .001$
scalar_prune_spawn vs hsap_full	1.93	-5.85%	1.14	$p < .001$	$p < .001$

Note. The Δ sign denotes the respective difference between the two models compared.

Convergence Efficiency

To address sub-RQ 4, I report the number of optimization steps and wall-clock time required to reach a target validation cross-entropy. Validation cross-entropy was evaluated every 10,000 steps, so time-to-quality is reported at a 10,000-step resolution. Under the criterion of validation cross-entropy < 3.50, *baseline* reached the threshold first at 150,000 steps (4.19 hours), followed by *hsap_full* at 160,000 steps (4.25 hours). *Baseline_prune* and *scalar_prune_spawn* did not reach validation cross-entropy < 3.50 within the logged training horizon. Because the 3.50 threshold excluded two models, I additionally report validation cross-entropy < 3.55 as criterion. Under this threshold, *hsap_full* and *baseline* reached the target at 130,000 steps, with *hsap_full* requiring 3.46 hours and *baseline* requiring 3.63 hours. *Baseline_prune* reached the target at 220,000 steps (4.15 hours), while *scalar_prune_spawn* reached it at 210,000 steps but required 4.59 hours.

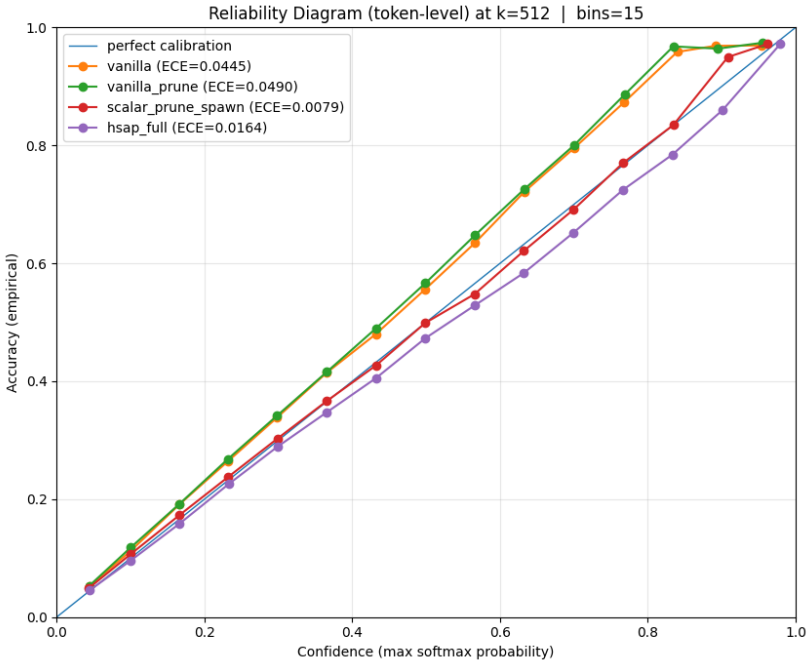
Error Analysis

Addressing Sub-RQ 5 on error analysis, I first evaluated token-level calibration at full context to assess probability quality. Figure 4 reports token-level calibration at full context.

Based on the reliability diagram and the expected calibration error, *baseline* and *baseline_prune* were mostly underconfident, with empirical accuracy exceeding confidence across much of the mid-confidence range, whereas *hsap_full* showed systematic overconfidence. *Scalar_prune_spawn* lay closest to the identity line and attained the lowest ECE at 0.009, followed by *hsap_full* at 0.015, *baseline* at 0.045, and *baseline_prune* at 0.051. This discrepancy between calibration and test loss underscores that improvements in perplexity do not always translate into better-calibrated probability estimates.

Figure 4

A Reliability Diagram of the Four Models

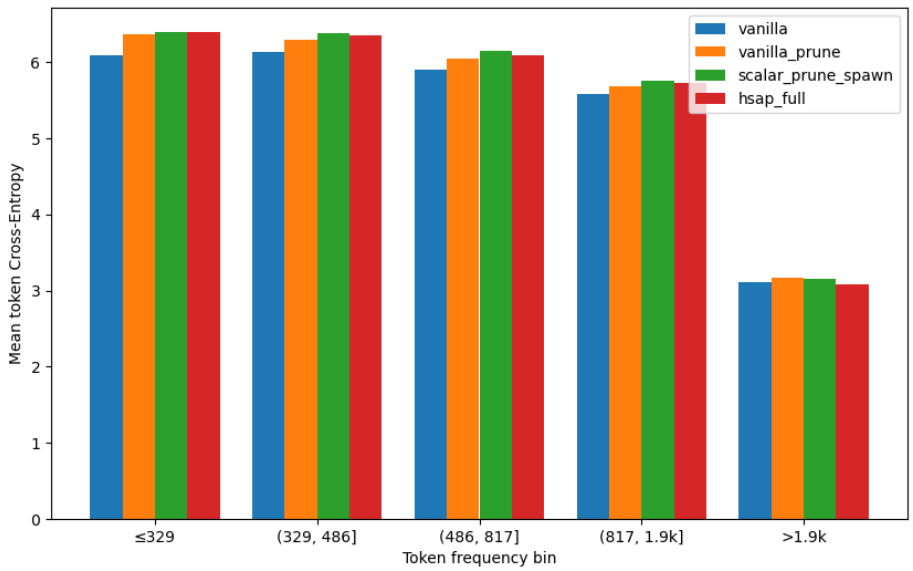


Following the calibration results, I examined how prediction difficulty varied with token rarity. Using five quantile-based token-frequency bins, Figure 5 shows that mean token cross-entropy decreased monotonically with frequency across all models. Across the four rarer bins up to 1.9k, *baseline* attained the lowest mean token loss, *hsap_full* scored slightly higher, and the other two models scored the highest. In the most frequent bin above 1.9k, *hsap_full* achieved the lowest mean token loss. This aligned with the test cross-entropies

because the overall mean is token-weighted and the most frequent bin contained a large share of test-set instances (Appendix C, Figure C6). Spearman rank correlations between token frequency and token-level loss were negative for all models: ρ = −.464 (*baseline*), ρ = −.463 (*baseline_prune*), ρ = −.462 (*scalar_prune_spawn*), and ρ = −.465 (*hsap_full*). The permutation tests indicated small differences in correlation magnitude (Holm-Bonferroni corrected): *baseline* differed significantly from *scalar_prune_spawn* ($p < .001$), and *hsap_full* differed significantly from *scalar_prune_spawn* ($p < .001$), whereas *baseline* did not differ from *hsap_full* ($p = .488$) or *baseline_prune* ($p = .158$).

Figure 5

The Mean Token Cross-Entropy Stratified by Quantile Bins

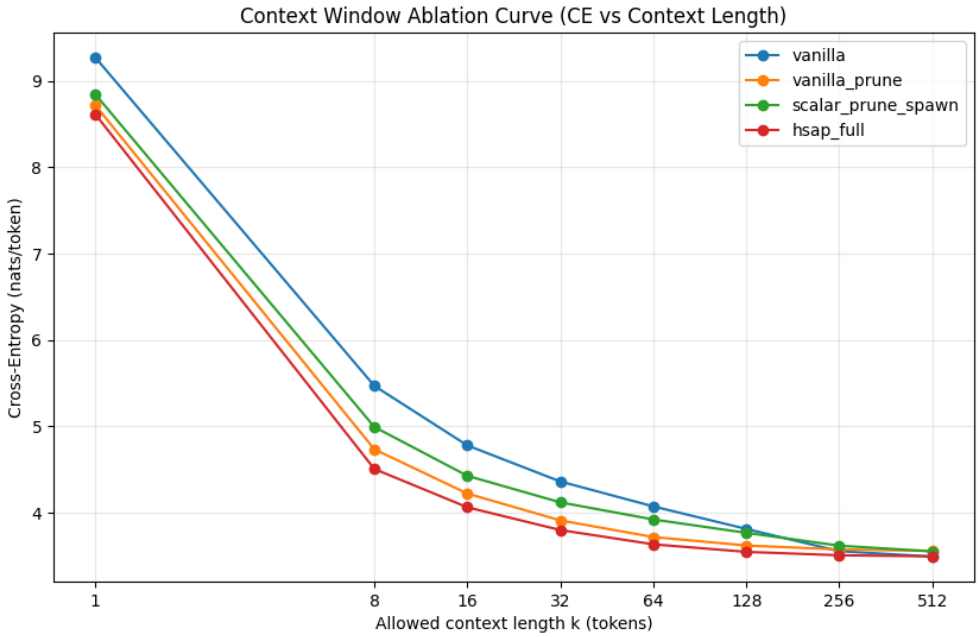


I also assessed how quickly models approach full-context performance as truncated context length *k* increases. Figure 7 shows cross-entropy decreased with *k*, with diminishing gains as *k* approached full context. Using the Effective Context Length at the 95% criterion, *hsap_full* and *baseline_prune* reached the threshold at $k = 64$, *scalar_prune_spawn* at $k = 128$, and *baseline* at $k = 256$. Consistent with these ECL values, *baseline* degraded most

under aggressive truncation, while *hsap_full* and *baseline_prune* retained more of their full-context performance at small *k*; differences narrow as *k* increases.

Figure 7

Context-Window Ablation: Increase in Cross-Entropy Relative to Log-Scaled Context Length

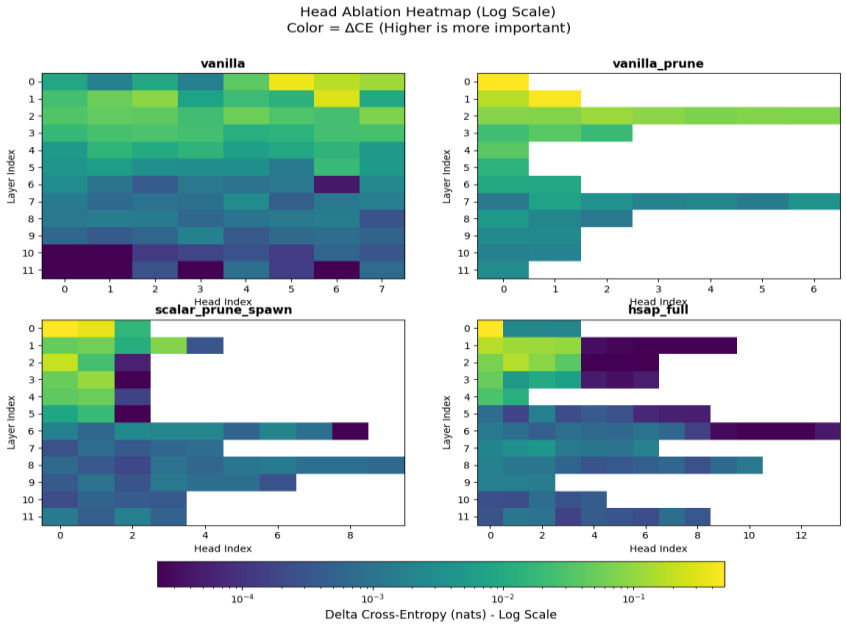


Ablation Analysis

To address Sub-RQ 6, I ablated one attention head at a time at inference and measured the resulting increase in cross-entropy. Figure 8 summarizes the results. Across models, sensitivity was highest in early layers and lower in later layers. The pruning-based variants showed a more uneven importance profile than the baseline Transformer, with larger ΔCE concentrated in a smaller subset of remaining heads, consistent with concentration of importance after pruning. This effect was strongest for the prune-only model, which removed heads without reallocating capacity. In HSAP, importance remained concentrated early but less sharply than in prune-only, suggesting a more distributed allocation of performance-critical attention.

Figure 8

Attention-Head Ablation Heatmaps for the Four Models



Note. Color encodes ΔCE on a log scale, where larger values indicate greater importance.

The next chapter interprets these findings in relation to the research questions, prior work, and limitations.

Discussion

Summary and Discussion of the Results

This thesis examined whether Hierarchical Spawn-and-Prune (HSAP) improves next-token prediction on WikiText-103 relative to a decoder-only baseline Transformer under a matched wall-clock compute budget. The following sub-research questions were answered:

1. Sub-RQ 1 (pruning-only): learned gating for pruning reduced test performance versus *baseline* with a large effect size ($\Delta CE \approx +0.067$; $p < .001$), indicating that sparsification without additional structure removes performance-critical capacity.

2. Sub-RQ 2 (spawn + prune): adding scalar-gated spawning produced a small but significant gain over pruning alone (ΔCE ≈ −0.006; Holm–Bonferroni-adjusted p = .036), but both sparse variants remained clearly worse than *baseline* (p < .001).
3. Sub-RQ 3 (HSAP): enforcing hierarchical specialization largely recovered the pruning-induced degradation; HSAP was statistically indistinguishable from *baseline* on test CE (ΔCE ≈ +0.004) while outperforming the pruning-only and prune+spawn variants (p < .001).
4. Sub-RQ 4 (efficiency): *baseline* reached a CE < 3.50 first (150,000 steps, 4.19 hours), with HSAP close behind (160,000, 4.25 hours). For a CE < 3.55, both reached the target at 130,000 steps, with HSAP slightly faster in wall-clock time (3.46 hours vs. 3.63 hours).
5. Sub-RQ 5 (error analysis): Performance differences were regime-dependent. In calibration analyses, HSAP was overconfident, whereas pruning-only and the *baseline* were underconfident. Although calibration error was lower for the gated models (ECE: prune+spawn 0.009; HSAP 0.015) than for *baseline* (ECE = 0.045), this did not translate into lower test cross-entropy. Token-frequency effects also varied across bins: *baseline* performed best in the less frequent token bins, while HSAP showed its strongest relative performance in the most frequent token bin. Under truncation, HSAP and pruning-only reached near-full performance at shorter context lengths, suggesting greater context efficiency when context is limited.
6. Sub-RQ 6 (ablations): across variants, critical computation concentrated in early layers and a small subset of heads. Pruning variants exhibited more uneven importance patterns, whereas HSAP's importance was more distributed across heads.

Interpretation of the Results

Under a fixed wall-clock compute budget, HSAP matches the baseline Transformer in

average test performance, but retains accuracy better as context length decreases. This supports the view that added architectural control need not reduce cross-entropy when training time is constant. The pruning-only model performs worst, consistent with the idea that aggressive pruning removes useful computation faster than remaining parameters can compensate. Adding spawning improves over pruning, but only marginally. Because the prune+spawn model and HSAP use identical pruning and spawning thresholds and sparsity strength, their comparison isolates hierarchical specialization from controller calibration. Under these shared settings, the prune+spawn model prunes at similar rates but retains fewer spawned heads, suggesting that specialization mainly improves the persistence and utility of reallocated capacity rather than the frequency of structural updates. Recovering capacity without stronger specialization therefore appears insufficient to restore baseline performance. However, the prune+spawn model reused HSAP controller settings, so it should be interpreted as an 'active-mechanism' ablation rather than a fully optimized prune+spawn baseline. The HSAP matches the baseline Transformer on full-context loss and outperforms the prune+spawn variant, implying that hierarchical specialization reallocates capacity more effectively than pruning and spawning alone.

Training efficiency depends on the evaluation criterion: under a stricter validation target, the baseline Transformer reaches the threshold first, whereas under a looser target, HSAP does so earlier. Thus, conclusions about time-to-quality vary with the chosen target, even under identical compute constraints.

Error analyses show that aggregate test loss conceals regime-level differences: calibration diverges sharply across variants, indicating that probability quality can differ from likelihood: a critical issue when downstream deploying a model. Token-frequency analysis further confirms the expected monotone frequency–loss relationship but reveals differing rankings across bins: HSAP performs best on frequent tokens, whereas the baseline

Transformer remains stronger on rarer ones. Context ablations suggest that hierarchical attention primarily enhances robustness, with HSAP reaching near-full performance at shorter context lengths, implying more efficient local-context use. The baseline Transformer benefits more from long-range information.

Head-ablation heatmaps show that performance-critical attention concentrates in early layers: pruning increases reliance on a few heads, while HSAP distributes importance more evenly among active heads.

Comparison to Prior Work

Test-set perplexities ranged from 32.89 (baseline Transformer) to 35.16 (prune-only), with intermediate values for the HSAP (33.00) and the prune+spawn variant (34.93). Comparing this with state-of-the-art WikiText-103 results confirms that none of the models approach the state-of-the-art: the Transformer-XL achieved 18.3 (Dai et al., 2019), adaptive-input models 18.7 (Baevski & Auli, 2019), and retrieval-augmented variants 17.6 and 16.08 (Yogatama et al., 2021; Alon et al., 2022). However, figures are not directly comparable because this study used a smaller backbone and different tokenization, context length, and training regimes. This study therefore insisted on controlled, compute-matched comparison rather than benchmark parity.

Prior pruning studies found that many heads can be removed with little loss (Michel et al., 2019; Voita et al., 2019). In contrast, the pruning-only variant here reduced performance by 6.46% ($p < .001$), likely due to the aggressive schedule leaving only 32 heads, limited tuning, and threshold sensitivity. Comparing HSAP with the prune+spawn model (Δ 5.85%, $p < .001$) shows that enforcing hierarchical structure benefits spawning, consistent with hierarchical-attention work reporting gains from structured specialization (Beltagy et al., 2020; Videau et al., 2025; Yu et al., 2023).

Regarding efficiency, HSAP reaches moderate quality sooner than the baseline

Transformer (CE < 3.55) but is slightly slower at stricter targets (CE < 3.50), consistent with conditional-computation work showing that routing/dispatch overhead can reduce or negate wall-clock gains depending on regime and implementation (Shazeer et al., 2017; Fedus et al., 2021).

HSAP modeled frequent tokens more accurately and degraded less under context truncation, but performed worse on rare tokens. Because frequent tokens require less context (Khandelwal et al., 2018), specialization may bias HSAP toward efficient local modeling. This aligns with findings that locality-biased attention captures nearby dependencies but misses long-range dependencies (Beltagy et al., 2020; Zaheer et al., 2021). As WikiText-103 contains strong long-range dependencies (Merity et al., 2016), this bias could limit HSAP's gains. Recent work also questions perplexity as a sufficient metric, proposing key-token-focused alternatives (Fang et al., 2025).

Calibration and next-token performance need not covary (Guo et al., 2017), so calibration differences reflect probability quality rather than overall modeling strength. Head and layer ablations align with prior findings that few heads dominate contribution (Michel et al., 2019; Voita et al., 2019) and that models are more robust to removing upper than lower layers (Fan et al., 2019; Sajjad et al., 2020).

Although pruning and hierarchical attention each show promise individually, their combination did not surpass the baseline Transformer under strict wall-clock matching. Three factors likely contribute: (1) amortization, as HSAP's control mechanisms require more tuning and stability; (2) criterion mismatch, where pruning and spawning signals may misalign with specialization; and (3) overhead, as dynamic updates can add runtime cost and disrupt optimization.

Scientific and Societal Impact

The central contribution of this thesis is a controlled, compute-matched evaluation of

HSAP against a strong decoder-only Transformer baseline on WikiText-103. It isolates the marginal effects of learned pruning, spawning, and hierarchical specialization under identical wall-clock constraints. The results show that pruning alone can significantly degrade next-token prediction when training time is fixed. They also show that spawning recovers only marginally unless it is coupled with hierarchical specialization. Scientifically, this refines the understanding of structured sparsity by suggesting that its benefit may lie less in improving full-context perplexity and more in shifting regime behavior, since HSAP retained performance better under reduced context length and exhibited distinct error characteristics across token-frequency bins. Practically, improved context efficiency is relevant for long-context deployment settings where inference cost and memory footprint are primary constraints. By maintaining task performance under a reduced effective context length, such methods can lower computational demand and energy expenditure per token, thereby improving the feasibility of widespread deployment of LLMs. Nevertheless, the absence of a full-context performance gain under strict wall-clock budgets cautions against deploying additional control mechanisms without sufficient tuning and stability controls.

Limitations and Future Directions

Several limitations constrain how fully the results answer the research questions. First, each configuration used a single random seed, limiting robustness and reducing confidence in observed small differences between variants across runs. Second, although the study includes staged ablations (prune-only, prune+spawn, HSAP), causal attribution remains difficult because the mechanisms interact. Third, the hyperparameter trial budget was limited, and the tuning budget did not match the final training budget, which can bias conclusions if variants differ in sensitivity to pruning thresholds, spawning schedules, or other controller settings. Fourth, conclusions may depend on tokenization, particularly for token-frequency analyses and token-level cross-entropy. Fifth, the prune+spawn variant controller

hyperparameters were not independently optimized in the final-run protocol, so its results may underestimate a fully tuned controller. Finally, WikiText-103 is a single benchmark with known idiosyncrasies and may not represent broader long-context or domain-diverse language modeling regimes.

Despite these constraints, the primary conclusions are valid within scope. Internal validity is strengthened by the controlled design: all variants share the same dataset, tokenization, training pipeline, and evaluation procedure, and are compared under a matched compute budget. The incremental ablation structure supports interpretation of the marginal effects of pruning, spawning, and hierarchical specialization, even if external generalization remains uncertain.

Future work should repeat experiments with multiple seeds and larger tuning budgets, test alternative tokenization schemes and datasets, and refine HSAP to better preserve global information. One direction is to enforce a mix of local and long-context specialists to avoid bias toward short-range dependencies. Also, training stability may improve with a grace period for newly spawned heads and alternative spawn criteria, such as conflicting gradients. Structured regularizers such as DropHead or LayerDrop may further increase robustness to component removal and yield more reliable head-importance estimates, improving pruning and spawning decisions.

Conclusion

Under a matched wall-clock compute budget, this thesis evaluated whether the Hierarchical Spawn-and-Prune (HSAP) would improve next-token prediction on WikiText-103 relative to a decoder-only Transformer baseline. The results show that HSAP does not outperform the baseline Transformer on average full-context test performance but consistently mitigates degradation introduced by learned pruning and improves robustness when context length is reduced, indicating that architectural complexity can shift regime

behavior even without lowering cross-entropy.

The central contribution of this work is a controlled, compute-matched ablation of pruning, spawning, and hierarchical specialization within one framework. It clarifies that pruning alone can remove performance-critical computation faster than remaining capacity can compensate, and that spawning provides only marginal recovery unless coupled with hierarchical specialization. Error analysis shows that average loss can obscure meaningful differences across frequency regimes and under context truncation.

These findings have practical implications for long-context language modeling: hierarchical specialization may be more valuable as a context-efficiency mechanism rather than a route to lower perplexity under strict compute constraints. Future research should test HSAP at larger compute and tuning budgets, across multiple random seeds and tokenization schemes, and incorporate mechanisms preserving global dependencies, such as long-context specialist heads and stability-aware spawning procedures.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)* (pp. 2623–2631). https://doi.org/10.1145/3292500.3330701

Alon, U., Xu, F. F., He, J., Sengupta, S., Roth, D., & Neubig, G. (2022). *Neuro-symbolic language modeling with automaton-augmented retrieval*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2201.12431

Baevski, A., & Auli, M. (2019). *Adaptive input representations for neural language modeling*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1809.10853

Behnke, M., & Heafield, K. (2020). Losing heads in the lottery: Pruning transformer attention in neural machine translation. In B. Webber, T. Cohn, Y. He, & Y. Liu (Eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 2664–2674). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.211

Beltagy, I., Peters, M. E., & Cohan, A. (2020). *Longformer: The long-document transformer*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2004.05150

Bian, S., Yu, T., Venkataraman, S., & Park, Y. (2025). *Scaling laws meet model architecture: Toward inference-efficient LLMs*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2510.18245

Braverman, M., Chen, X., Kakade, S. M., Narasimhan, K., Zhang, C., & Zhang, Y. (2019). *Calibration, entropy rates, and memory in language models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1906.05664

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child,

R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., … Amodei, D. (2020). *Language models are few-shot learners*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2005.14165

Caballero, E., Gupta, K., Rish, I., & Krueger, D. (2022). *Broken neural scaling laws*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2210.14891

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.

Chen, J., Li, C., Yuan, Y., & Yao, A. C. (2025). *Hierarchical attention generates better proofs*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2504.19188

Chen, W., Xu, X., Han, X., Lin, Y., Xie, R., Liu, Z., Sun, M., & Zhou, J. (2023). *Boosting inference efficiency: Unleashing the power of parameter-shared pre-trained language models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2310.12818

da Costa-Luis, C. O. (2019). tqdm: A fast, extensible progress meter for Python and CLI. *Journal of Open Source Software, 4*(37), 1277. https://doi.org/10.21105/joss.01277

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). *Transformer-XL: Attentive language models beyond a fixed-length context*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1901.02860

Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., Zoph, B., Fedus, L., Bosma, M., Zhou, Z., Wang, T., Wang, Y. E., Webster, K., Pellat, M., Robinson, K., … Cui, C. (2021). *GLaM: Efficient scaling of language models with mixture-of-experts*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2112.06905

Fan, A., Grave, E., & Joulin, A. (2019). *Reducing transformer depth on demand with structured dropout*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1909.11556

Fang, L., Wang, Y., Liu, Z., Zhang, C., Jegelka, S., Gao, J., Ding, B., & Wang, Y. (2025). *What is wrong with perplexity for long-context language modeling?* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2410.23771

Fedus, W., Zoph, B., & Shazeer, N. (2021). *Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2101.03961

Good, P. I. (2005). *Permutation, parametric, and bootstrap tests of hypotheses* (3rd ed.). Springer.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). *On calibration of modern neural networks.* In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). *Array programming with NumPy*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2006.10256

He, H., & Su, W. J. (2024). *A law of next-token prediction in large language models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2408.13442

Hendrycks, D., & Gimpel, K. (2016). *Gaussian error linear units (GELUs)*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1606.08415

Hestness, J., Narang, S., Ardalani, N., Diamos, G., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., & Zhou, Y. (2017). *Deep learning scaling is predictable, empirically*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1712.00409

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., … Sifre, L. (2022). *Training compute-optimal large language models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2203.15556

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of*

Statistics, 6(2), 65–70. https://www.jstor.org/stable/4615733

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Jurafsky, D., & Martin, J. H. (2025). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd ed.; online manuscript released August 24, 2025) [Manuscript]. https://web.stanford.edu/~jurafsky/slp3/

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling laws for neural language models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2001.08361

Khandelwal, U., He, H., Qi, P., & Jurafsky, D. (2018). *Sharp nearby, fuzzy far away: How neural language models use context*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1805.04623

Kudo, T., & Richardson, J. (2018). *SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1808.06226

Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs. *Frontiers in Psychology, 4*, Article 863. https://doi.org/10.3389/fpsyg.2013.00863

Lhoest, Q., Villanova del Moral, A., Jernite, Y., Thakur, A., von Platen, P., Patil, S., Chaumond, J., Drame, M., Plu, J., Tunstall, L., Davison, J., Šaško, M., Chhablani, G., Malik, B., Brandeis, S., Le Scao, T., Sanh, V., Xu, C., Patry, N., … Wolf, T. (2021). *Datasets: A community library for natural language processing*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2109.02846

Li, B., Wang, Z., Huang, S., Bragin, M., Li, J., & Ding, C. (2023). Towards lossless head pruning through automatic peer distillation for language models. In E. Elkind (Ed.), *Proceedings of*

the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23) (pp. 5113–5121). International Joint Conferences on Artificial Intelligence Organization. https://doi.org/10.24963/ijcai.2023/568

Li, J., Cotterell, R., & Sachan, M. (2021). *Differentiable subset pruning of transformer heads.* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2108.04657

Lin, P., Zhang, Z., & Xu, Z.-Q. J. (2025). *Reasoning bias of next token prediction training*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2502.02007

Liu, Y., Wu, Y.-H., Sun, G., Zhang, L., Chhatkuli, A., & Van Gool, L. (2024). Vision transformers with hierarchical attention. *Machine Intelligence Research, 21*, 670–683. https://doi.org/10.1007/s11633-024-1393-8

Loshchilov, I., & Hutter, F. (2016). *SGDR: Stochastic gradient descent with warm restarts*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1608.03983

Loshchilov, I., & Hutter, F. (2019). *Decoupled weight decay regularization*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1711.05101

Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). *Pointer sentinel mixture models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1609.07843

Michel, P., Levy, O., & Neubig, G. (2019). Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems* (Vol. 32, pp. 14014–14024). https://proceedings.neurips.cc/paper_files/paper/2019/file/2c601ad9d2ff9bc8b282670cdd54f69f-Paper.pdf

Nickolls, J., Buck, I., Garland, M., & Skadron, K. (2008). Scalable parallel programming with CUDA. *Queue, 6*(2), 40–53. https://doi.org/10.1145/1401132.1401152

Oeberst, A., von der Beck, I., Matschke, C., Ihme, T. A., & Cress, U. (2020). Collectively biased representations of the past: Ingroup bias in Wikipedia articles about intergroup conflicts. *British Journal of Social Psychology, 59*(4), 791–818. https://doi.org/10.1111/bjso.12356

OpenAI. (2026). *ChatGPT* (January 12, 2026 version) [Large language model]. https://chatgpt.com/

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1912.01703

Patterson, D. A., Gonzalez, J., Le, Q. V., Liang, C., Munguia, L.-M., Rothchild, D., So, D. R., Texier, M., & Dean, J. (2021). *Carbon emissions and large neural network training*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2104.10350

Politis, D. N., & Romano, J. P. (1992). A circular block-resampling procedure for stationary data. In R. LePage & L. Billard (Eds.), *Exploring the limits of bootstrap* (pp. 263–270). Wiley.

Python Software Foundation. (2024). *Python* (Version 3.11.9) [Computer software]. https://www.python.org/

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners* [Technical report]. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Sajjad, H., Dalvi, F., Durrani, N., & Nakov, P. (2020). *On the effect of dropping layers of pre-trained transformer models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2004.03844

Sathe, A., & Sarawagi, S. (2024). *Efficient training of language models with compact and consistent next token distributions*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2407.02819

Schmahl, K. G., Viering, T. J., Makrodimitris, S., Naseri Jahfari, A., Tax, D., & Loog, M. (2020). Is Wikipedia succeeding in reducing gender bias? Assessing changes in gender bias in Wikipedia using word embeddings. In *Proceedings of the Fourth Workshop on Natural Language Processing and Computational Social Science* (pp. 94–103). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.nlpcss-1.11

Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural machine translation of rare words with subword units*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1508.07909

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). *Outrageously large neural networks: The sparsely-gated mixture-of-experts layer*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1701.06538

Sorscher, B., Geirhos, R., Shekhar, S., Ganguli, S., & Morcos, A. S. (2022). *Beyond neural scaling laws: Beating power law scaling via data pruning*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2206.14486

Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology, 15*(1), 72–101. https://doi.org/10.2307/1412159

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

Strubell, E., Ganesh, A., & McCallum, A. (2019). *Energy and policy considerations for deep learning in NLP*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1906.02243

Tack, J., Lanchantin, J., Yu, J., Cohen, A., Kulikov, I., Lan, J., Hao, S., Tian, Y., Weston, J., & Li, X. (2025). *LLM pretraining with continuous concepts*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2502.08524

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1706.03762

Videau, M., Youbi Idrissi, B., Leite, A., Schoenauer, M., Teytaud, O., & Lopez-Paz, D. (2025). *From bytes to ideas: Language modeling with autoregressive U-Nets*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2506.14761

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E.,

Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). *Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1905.09418

Wang, W., Xie, E., Li, X., Fan, D.-P., Song, K., Liang, D., Lu, T., Luo, P., & Shao, L. (2021). *Pyramid vision transformer: A versatile backbone for dense prediction without convolutions*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2102.12122

Wang, X., Zhang, X., Luo, Z., Sun, Q., Cui, Y., Wang, J., Zhang, F., Wang, Y., Li, Z., Yu, Q., Zhao, Y., Ao, Y., Min, X., Li, T., Wu, B., Zhao, B., Zhang, B., Wang, L., Liu, G., … Wang, Z. (2024). *Emu3: Next-token prediction is all you need*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2409.18869

Wikimedia Foundation. (2025). *Community Insights 2024 report* [Report]. Meta-Wiki. https://meta.wikimedia.org/w/index.php?title=Community_Insights/Community_Insights_2024_Report&oldid=29325900

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T.-Y. (2020). *On layer normalization in the Transformer architecture*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2002.04745

Yogatama, D., de Masson d'Autume, C., & Kong, L. (2021). *Adaptive semiparametric language models*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2102.02557

Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., & Lewis, M. (2023). *MEGABYTE: Predicting million-byte sequences with multiscale transformers*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2305.07185

Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontañón, S., Pham, P., Ravula, A., Wang, Q., Yang, L., & Ahmed, A. (2021). *Big Bird: Transformers for longer sequences*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2007.14062

Zhu, Z., & Soricut, R. (2021). *H-Transformer-1D: Fast one-dimensional hierarchical attention for sequences*. [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2107.11906

Zipf, G. K. (1949). *Human behavior and the principle of least effort: An introduction to human ecology.* Addison-Wesley.

Appendix A

Notation

For the Methodology section, we define some terms:

- L: number of Transformer layers.
- l: index of a layer, l in $\{1, \dots, \mathrm{L}\}$.
- H_l: number of attention heads in layer l.
- i: index of a head within layer l, i $\{1, \dots, \mathrm{H_l}\}$.
- $h = (l, i)$: index of a specific head.
- $x_l \in \mathbb{R}^{T_{\text{seq}} \times d_{\text{model}}}$: input sequence to layer l(output of the previous layer), where T_{seq} is the sequence length.
- d_{model}: Transformer model dimension (hidden size).
- $\mathrm{f_h}(\mathrm{x_l}; \theta_\mathrm{h})$: output of attention head h given input $\mathrm{x_l}$, parameterized by θ_h. We include the head output projection in $f_h$so that $f_h(x_l; \theta_h) \in \mathbb{R}^{T_{\text{seq}} \times d_{\text{model}}}$, making the gated sum well-defined.
- $y_{l(x_l)}$: output of the gated attention sublayer in layer l, defined as: $y_l(x_l) = \sum_{i=1}^{H_l} g_{l,i}\ f_{l,i}(x_l; \theta_{l,i})$.
- a_h: the scalar parameter associated with head h.
- $g_h \in (0,1)$: scalar gate for head h, defined as $g_h = \sigma(a_h)$.
- Given $h = (l, i)$, we use $g_h \equiv g_{l,i}$ and $f_h \equiv f_{l,i}$.
- $\sigma(\cdot)$: logistic sigmoid function used for mapping $\mathrm{a_h}$ to $\mathrm{g_h}$.
- $\mathrm{D} = \{(\mathrm{x^j}, \mathrm{y^j})\}_{\{\mathrm{j}=1\}}^{\mathrm{N}}$: training dataset of N examples.
- $\mathrm{f_{model(x)}}$: full Transformer model.
- $\mathrm{l_{task(y_{hat}, y)}}$: task loss (cross-entropy) between prediction $\mathrm{y_{hat}}$ and target y.
- $\mathrm{L_{total}}$: total loss including both task loss *and* sparsity regularization.

- $\text{lambda}_{\text{sparse}}$: non-negative coefficient determining the strength of the sparsity penalty on the gates.
- T: the set of all attention heads currently present.
- v_h: within-layer normalized gate score (defined in the Methodology section)
- $\text{tau}_{\text{prune}}$: gate threshold used for pruning heads.
- $\text{tau}_{\text{spawn}}$: gate threshold used for spawning child heads, with $\text{tau}_{\text{spawn}} >= \text{tau}_{\text{prune}}$.
- K: number of structural steps.
- $\text{S} = \{\text{s}_1, \ldots, \text{s}_\text{K}\}$: set of global training steps at which pruning and spawning are applied.
- $P_l^{(s_k)}$: set of heads in layer l that are pruned at structural step s_k.
- $A_l^{(s_k)}$: set of active heads in layer l immediately after structural step s_k.
- $Spawn_l^{(s_k)}$: subset of active heads in layer l at step s_k that will spawn children.
- $h_{\text{child}}(h)$: the child head spawned from parent head h.
- ϵ: small perturbation used to initialize the parameters of the child head.
- g_{init}: small positive initial gate value used for child heads.
- $\text{parent}(h)$: parent head of h in the hierarchy.
- t: token position index in the sequence.
- d_{head}: dimensionality of the query/key/value vectors for a single head.
- $q_{l,h,t} \in \mathbb{R}^{d_{\text{head}}}$: query vector from head h at token position t.
- $c_h \in \mathbb{R}^{d_{\text{head}}}$: learnable center (prototype) of head h in query space.
- $\sigma_h > 0$: width parameter of the Gaussian gate for head h.
- $\sigma_{\min} > 0$: fixed minimum width used to clamp σ_h in the implementation.
- $w_{l,h,t} \in (0,1]$: Gaussian gate for head h at token t.
- $y_{l,h,t}$: standard attention output of head h at token t (before gating).

- $\tilde{y}_{l,h,t}$: gated output of head h at token t.
- $\tilde{y}_{l,h,t} := \coloneqq g_h \, w_{l,h,t} \, y_{l,h,t}$.

Appendix B

Model Card 1: Baseline Transformer

Model identity

Model name: Baseline Transformer

Variant: Baseline decoder-only Transformer

Author / affiliation: Jonathan Otterspeer, MSc Data Science & Society, Tilburg University

Primary task: Next-token prediction (causal language modeling) on WikiText-103

Architecture

Model type: Decoder-only Transformer language model

Decoder layers (L): 12

Hidden size (d_model): 512

Feed-forward dimension (d_ff): 2,048

Attention heads: 8 heads per layer (fixed)

Maximum sequence length: 512

Positional encoding: Learned absolute positional embeddings (length 512)

Layer normalization: Pre-LayerNorm with residual connections around attention and feed-forward sublayers

FFN activation: GELU

Training data

Dataset: WikiText-103 (configuration: wikitext-103-raw-v1)

Source & license: Curated by Salesforce Research from English Wikipedia Good and Featured articles; CC BY-SA 3.0

Train / validation / test: Train: 1,801,350 segments (103,227,021 tokens); Validation: 3,760 (217,646); Test: 4,358 (245,569)

Tokenization & sequencing: SentencePiece subword tokenization (vocabulary size 32,000).

EOS appended after each original segment; continuous-stream tokenization split into non-overlapping 512-token blocks (no padding).

Other preprocessing: None

Training procedure

Frameworks: Python 3.11.9; PyTorch 2.10; Hugging Face Datasets 4.4.1; SentencePiece 0.2.1; Optuna 4.6.0; SciPy 1.16.3; Matplotlib 3.10.7

Hardware: Single NVIDIA RTX 5070 GPU (12 GB VRAM)

Optimizer: AdamW

Batch size: 8

Compute budget (tuning): 20 Optuna trials per variant; 1.5 hours wall-clock per trial

Compute budget (final training): 6 hours wall-clock per variant

Random seed: 123

Hyperparameters

Fixed backbone: L=12, d_model=512, d_ff=2,048; initial heads=8, sequence length=512, GELU; pre-LayerNorm

Tuned with Optuna (optimal values):

- Learning rate: 9.848×10^{-4}
- Weight decay: 8.164×10^{-3}
- Gradient clipping: 1.434
- Dropout: 4.739×10^{-3}
- Label smoothing: 1.439×10^{-1}
- Warm-up steps: 6,000

Evaluation

Primary metrics: Cross-entropy (negative log-likelihood) and perplexity on the test set

Validation protocol: Validation every 10,000 optimization steps; time-to-quality defined as the first step where validation cross-entropy < 3.5

Statistical tests: Paired block-level permutation tests on non-overlapping 512-token test blocks (10,000 permutations) with Holm–Bonferroni correction at α = .05; circular moving-block bootstrap (10,000 resamples; block length 512 tokens) for 95% confidence intervals

Effect sizes: Relative perplexity improvement (%) and paired Cohen's $d_{(z)}$ (computed on block-level cross-entropy differences)

Error analysis: Token-frequency regime analysis (Zipf plot; quantile frequency bins; Spearman rank correlation between token frequency and token-level loss; permutation tests for pairwise differences). Calibration (reliability diagrams; expected calibration error, ECE).

Context sensitivity (context-length ablation; Effective Context Length at the 95% criterion).

Ablations: Head ablations by setting individual head outputs to zero; layer ablations approximated by zeroing the outputs of all attention heads within a layer

Model Card 2: Pruning-only Variant

Model identity

Model name: Pruning-only Transformer

Variant: Scalar head gates + $L1$ sparsity + pruning (no spawning, no Gaussian specialization)

Author / affiliation: Jonathan Otterspeer, MSc Data Science & Society, Tilburg University

Primary task: Next-token prediction (causal language modeling) on WikiText-103

Architecture

Model type: Decoder-only Transformer language model

Decoder layers (L): 12

Hidden size (d_model): 512

Feed-forward dimension (d_ff): 2,048

Attention heads: 8 heads per layer at initialization, head count changes via pruning/spawning

during training

Maximum sequence length: 512

Positional encoding: Learned absolute positional embeddings (length 512)

Layer normalization: Pre-LayerNorm with residual connections around attention and feed-forward sublayers

FFN activation: GELU

Pruning / spawning mechanisms

Scalar head gates: One learned scalar gate per head; attention sublayer output is a gated sum of head outputs

Sparsity regularization: $L1$ penalty on scalar gates ($lambda\_sparse$)

Structural updates: Applied at fixed intervals (structural step interval)

Pruning rule: Heads with within-layer normalized gate below tau_prune are removed

Training data

Dataset: WikiText-103 (configuration: wikitext-103-raw-v1)

Source & license: Curated by Salesforce Research from English Wikipedia Good and Featured articles; CC BY-SA 3.0

Train / validation / test: Train: 1,801,350 segments (103,227,021 tokens); Validation: 3,760 (217,646); Test: 4,358 (245,569)

Tokenization & sequencing: SentencePiece subword tokenization (vocabulary size 32,000). EOS appended after each original segment; continuous-stream tokenization split into non-overlapping 512-token blocks (no padding).

Other preprocessing: None (beyond tokenization/segmentation)

Training procedure

Frameworks: Python 3.11.9; PyTorch 2.10; Hugging Face Datasets 4.4.1; SentencePiece 0.2.1; Optuna 4.6.0; SciPy 1.16.3; Matplotlib 3.10.7

Hardware: Single NVIDIA RTX 5070 GPU (12 GB VRAM)

Optimizer: AdamW

Learning-rate schedule: Warm-up (4,000 steps) followed by cosine decay

Batch size: 8

Compute budget (tuning): 20 Optuna trials per variant; 1.5 hours wall-clock per trial

Compute budget (final training): 6 hours wall-clock per variant

Implementation detail: Pruning/spawning is implemented by shrinking and rebuilding packed QKV and output projection matrices to reflect the current head count (mask-free), so pruning yields actual compute savings.

Random seed: 123

Hyperparameters

Fixed backbone: L=12; d_model=512; dff=2,048; initial heads=8; sequence length=512; GELU; pre-LayerNorm

Tuned with Optuna (optimal values):

- Learning rate: 9.671×10^{-4}
- Weight decay: 1.516×10^{-5}
- Gradient clipping: 8.003×10^{-1}
- Dropout: 1.070×10^{-2}
- Label smoothing: 1.637×10^{-1}
- Warm-up steps: 4,000
- Sparsity strength: 2.554×10^{-4}
- Pruning threshold: 5.977×10^{-1}
- Structural step interval: 6,000

Evaluation

Primary metrics: Cross-entropy (negative log-likelihood) and perplexity on the test set

Validation protocol: Validation every 10,000 optimization steps; time-to-quality defined as the first step where validation cross-entropy < 3.5

Statistical tests: Paired block-level permutation tests on non-overlapping 512-token test blocks (10,000 permutations) with Holm–Bonferroni correction at α = .05; circular moving-block bootstrap (10,000 resamples; block length 512 tokens) for 95% confidence intervals

Effect sizes: Relative perplexity improvement (%) and paired Cohen's $d_{(z)}$ (computed on block-level cross-entropy differences)

Error analysis: Token-frequency regime analysis (Zipf plot; quantile frequency bins; Spearman rank correlation between token frequency and token-level loss; permutation tests for pairwise differences). Calibration (reliability diagrams; expected calibration error, ECE). Context sensitivity (context-length ablation; Effective Context Length at the 95% criterion).

Ablations: Head ablations by setting individual head outputs to zero; layer ablations approximated by zeroing the outputs of all attention heads within a layer

Model Card 3: Pruning + Spawning Variant

Model identity

Model name: Pruning + Spawning Transformer

Variant: Scalar head gates + $L1$ sparsity + pruning + spawning (no Gaussian specialization)

Author / affiliation: Jonathan Otterspeer, MSc Data Science & Society, Tilburg University

Primary task: Next-token prediction (causal language modeling) on WikiText-103

Architecture

Model type: Decoder-only Transformer language model

Decoder layers (L): 12

Hidden size (d_model): 512

Feed-forward dimension (d_ff): 2,048

Attention heads: 8 heads per layer at initialization, head count changes via pruning/spawning during training

Heads-per-layer cap: 16

Maximum sequence length: 512

Positional encoding: Learned absolute positional embeddings (length 512)

Layer normalization: Pre-LayerNorm with residual connections around attention and feed-forward sublayers

FFN activation: GELU

Pruning / spawning mechanisms

Scalar head gates: One learned scalar gate per head; attention sublayer output is a gated sum of head outputs

Sparsity regularization: $L1$ penalty on scalar gates ($lambda\_sparse$)

Structural updates: Applied at fixed intervals (structural step interval)

Pruning rule: Heads with within-layer normalized gate below tau_prune are removed

Spawning rule: Heads with within-layer normalized gate at or above tau_spawn spawn child heads

Child initialization: Children are initialized from the parent plus a small perturbation; child gates start at g_init.

Training data

Dataset: WikiText-103 (configuration: wikitext-103-raw-v1)

Source & license: Curated by Salesforce Research from English Wikipedia Good and Featured articles; CC BY-SA 3.0

Train / validation / test: Train: 1,801,350 segments (103,227,021 tokens); Validation: 3,760 (217,646); Test: 4,358 (245,569)

Tokenization & sequencing: SentencePiece subword tokenization (vocabulary size 32,000). EOS appended after each original segment; continuous-stream tokenization split into non-overlapping 512-token blocks (no padding).

Other preprocessing: None (beyond tokenization/segmentation)

Training procedure

Frameworks: Python 3.11.9; PyTorch 2.10; Hugging Face Datasets 4.4.1; SentencePiece 0.2.1; Optuna 4.6.0; SciPy 1.16.3; Matplotlib 3.10.7

Hardware: Single NVIDIA RTX 5070 GPU (12 GB VRAM)

Optimizer: AdamW

Learning-rate schedule: Warm-up (6,000 steps) followed by cosine decay

Batch size: 8

Compute budget (tuning): 20 Optuna trials per variant; 1.5 hours wall-clock per trial

Compute budget (final training): 6 hours wall-clock per variant

Implementation detail: Pruning/spawning is implemented by shrinking and rebuilding packed QKV and output projection matrices to reflect the current head count (mask-free), so pruning yields actual compute savings.

Random seed: 123

Hyperparameters

Fixed backbone: L=12; d_model=512; d_ff=2,048; initial heads=8; sequence length=512; GELU; pre-LayerNorm

Tuned with Optuna (optimal values):

- Learning rate: 5.752×10^{-4}
- Weight decay: 8.355×10^{-4}
- Gradient clipping: 8.798×10^{-1}

- Dropout: 9.458×10^{-2}
- Label smoothing: 8.051×10^{-2}
- Warm-up steps: 6,000
- Sparsity strength: 1.163×10^{-6}
- Pruning threshold: 2.448×10^{-1}
- Structural step interval: 6,000
- Spawning threshold: 7.039×10^{-1}
- Layer head cap: 16
- Child perturbation: 5.024×10^{-2}
- Child initial gate: 5.976×10^{-2}

Note. The eventual model used the spawning threshold, pruning threshold and sparsity strength of the HSAP model.

Evaluation

Primary metrics: Cross-entropy (negative log-likelihood) and perplexity on the test set

Validation protocol: Validation every 10,000 optimization steps; time-to-quality defined as the first step where validation cross-entropy < 3.5

Statistical tests: Paired block-level permutation tests on non-overlapping 512-token test blocks (10,000 permutations) with Holm–Bonferroni correction at $\alpha = .05$; circular moving-block bootstrap (10,000 resamples; block length 512 tokens) for 95% confidence intervals

Effect sizes: Relative perplexity improvement (%) and paired Cohen's $d_{(z)}$ (computed on block-level cross-entropy differences)

Error analysis: Token-frequency regime analysis (Zipf plot; quantile frequency bins; Spearman rank correlation between token frequency and token-level loss; permutation tests for pairwise differences). Calibration (reliability diagrams; expected calibration error, ECE). Context sensitivity (context-length ablation; Effective Context Length at the 95% criterion).

Ablations: Head ablations by setting individual head outputs to zero; layer ablations approximated by zeroing the outputs of all attention heads within a layer

Model Card 4: Full HSAP Variant

Model identity

Model name: HSAP Transformer

Variant: Scalar head gates + $L1$ sparsity + pruning + spawning + Gaussian specialization

Author / affiliation: Jonathan Otterspeer, MSc Data Science & Society, Tilburg University

Primary task: Next-token prediction (causal language modeling) on WikiText-103

Architecture

Model type: Decoder-only Transformer language model

Decoder layers (L): 12

Hidden size (d_model): 512

Feed-forward dimension (d_ff): 2,048

Attention heads: 8 heads per layer at initialization, head count changes via pruning/spawning during training

Heads-per-layer cap: 24

Maximum sequence length: 512

Positional encoding: Learned absolute positional embeddings (length 512)

Layer normalization: Pre-LayerNorm with residual connections around attention and feed-forward sublayers

FFN activation: GELU

Pruning / spawning mechanisms

Scalar head gates: One learned scalar gate per head; attention sublayer output is a gated sum of head outputs

Sparsity regularization: $L1$ penalty on scalar gates ($lambda\_sparse$)

Structural updates: Applied at fixed intervals (structural step interval)

Pruning rule: Heads with within-layer normalized gate below tau_prune are removed

Spawning rule: Heads with within-layer normalized gate at or above tau_spawn spawn child heads

Child initialization: Children are initialized from the parent plus a small perturbation; child gates start at g_init

Gaussian specialization gate: Token-dependent Gaussian gate on each head (prototype + width) to encourage specialization, child widths shrink relative to parent

Training data

Dataset: WikiText-103 (configuration: wikitext-103-raw-v1)

Source & license: Curated by Salesforce Research from English Wikipedia Good and Featured articles; CC BY-SA 3.0

Train / validation / test: Train: 1,801,350 segments (103,227,021 tokens); Validation: 3,760 (217,646); Test: 4,358 (245,569)

Tokenization & sequencing: SentencePiece subword tokenization (vocabulary size 32,000). EOS appended after each original segment; continuous-stream tokenization split into non-overlapping 512-token blocks (no padding).

Other preprocessing: None (beyond tokenization/segmentation)

Training procedure

Frameworks: Python 3.11.9; PyTorch 2.10; Hugging Face Datasets 4.4.1; SentencePiece 0.2.1; Optuna 4.6.0; SciPy 1.16.3; Matplotlib 3.10.7

Hardware: Single NVIDIA RTX 5070 GPU (12 GB VRAM)

Optimizer: AdamW

Learning-rate schedule: Warm-up (8,000 steps) followed by cosine decay

Batch size: 8

Compute budget (tuning): 20 Optuna trials per variant; 1.5 hours wall-clock per trial

Compute budget (final training): 6 hours wall-clock per variant

Implementation detail: Pruning/spawning is implemented by shrinking and rebuilding packed QKV and output projection matrices to reflect the current head count (mask-free), so pruning yields actual compute savings.

Random seed: 123

Hyperparameters

Fixed backbone: L=12; d_model=512; d_ff=2,048; initial heads=8; sequence length=512; GELU; pre-LayerNorm

Tuned with Optuna (optimal values):

- Learning rate: 9.671×10^{-4}
- Weight decay: 2.679×10^{-5}
- Gradient clipping: 1.898
- Dropout: 3.425×10^{-2}
- Label smoothing: 1.085×10^{-2}
- Warm-up steps: 8,000
- Sparsity strength: 3.663×10^{-4}
- Pruning threshold: 5.977×10^{-1}
- Structural step interval: 4,000 (×4=16,000 for the final run)
- Spawning threshold: 9.824×10^{-1}
- Layer head cap: 24
- Child perturbation: 1.085×10^{-1}
- Child initial gate: 7.916×10^{-2}
- Gaussian shrink factor: 6.988×10^{-1}

Evaluation

Primary metrics: Cross-entropy (negative log-likelihood) and perplexity on the test set

Validation protocol: Validation every 10,000 optimization steps; time-to-quality defined as the first step where validation cross-entropy < 3.5

Statistical tests: Paired block-level permutation tests on non-overlapping 512-token test blocks (10,000 permutations) with Holm–Bonferroni correction at $\alpha = .05$; circular moving-block bootstrap (10,000 resamples; block length 512 tokens) for 95% confidence intervals

Effect sizes: Relative perplexity improvement (%) and paired Cohen's $d_{(z)}$ (computed on block-level cross-entropy differences)

Error analysis: Token-frequency regime analysis (Zipf plot; quantile frequency bins; Spearman rank correlation between token frequency and token-level loss; permutation tests for pairwise differences). Calibration (reliability diagrams; expected calibration error, ECE). Context sensitivity (context-length ablation; Effective Context Length at the 95% criterion).

Ablations: Head ablations by setting individual head outputs to zero; layer ablations approximated by zeroing the outputs of all attention heads within a layer

Appendix C

Figure C6

The Distribution of Tokens across Frequency Bins